STAR RX72N - rx_encoder Library

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Test List

Global rx_encoder_init (const rx_encoder_config_t *config)

test_rx_mtu_encoder.c::test_encoder_init() Unit test coverage

Global rx_encoder_read_count (const rx_mtu_channel_t channel, rx_encoder_state_t *state)

test_rx_mtu_encoder.c::test_encoder_overflow() Tests overflow detection

Chapter 2

Directory Hierarchy

2.1 Directories

Chapter 3

File Index

3.1 File List

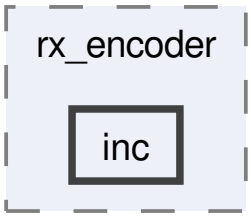
Here is a list of all files with brief descriptions:

Chapter 4

Directory Documentation

4.1 libs/rx_encoder/inc Directory Reference

Directory dependency graph for inc:

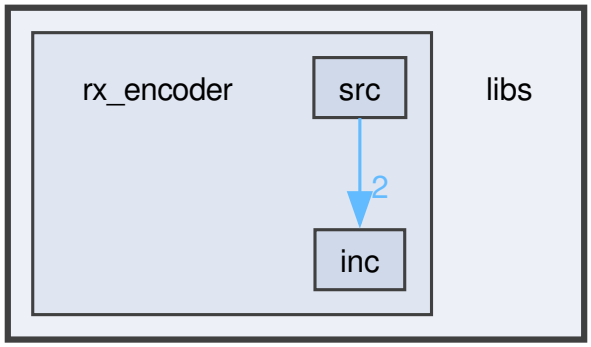


Files

- file rx_encoder_tpu.h
  TPU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode).
- file rx_mtu_encoder.h
  MTU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode).

4.2 libs Directory Reference

Directory dependency graph for libs:

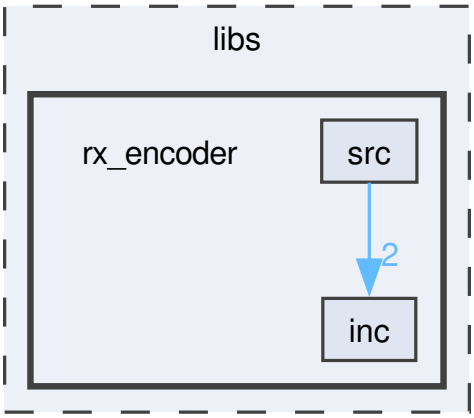


Directories

- directory rx_encoder

4.3 libs/rx_encoder Directory Reference

Directory dependency graph for rx_encoder:

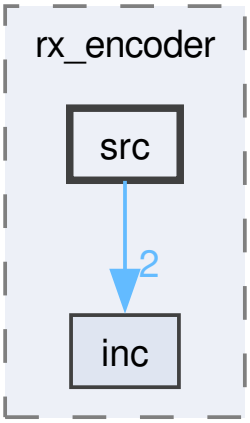


Directories

- directory inc
- directory src

4.4 libs/rx_encoder/src Directory Reference

Directory dependency graph for src:



Files

- file rx_encoder_tpu.c
  TPU Quadrature Encoder Driver Implementation (Phase Counting Mode).
- file rx_mtu_encoder.c
  RX72N MTU Quadrature Encoder Driver Implementation (Phase Counting Mode).

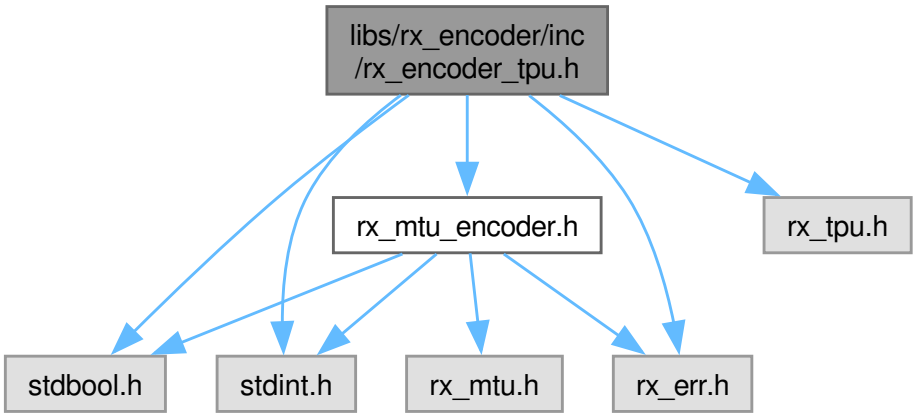
Chapter 5

File Documentation

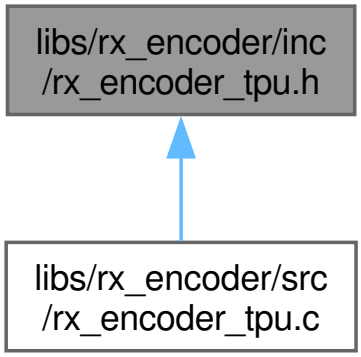
5.1 libs/rx_encoder/inc/rx_encoder_tpu.h File Reference

TPU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode).

#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
#include "rx_mtu_encoder.h"
#include "rx_tpu.h"
Include dependency graph for rx_encoder_tpu.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct rx_tpu_encoder_config_t
  TPU encoder configuration structure.

Functions

- rx_err_t rx_tpu_encoder_init (const rx_tpu_encoder_config_t *config)
  Initialize TPU channel for hardware quadrature encoder counting.
- rx_err_t rx_tpu_encoder_read_raw (rx_tpu_channel_t channel, uint16_t *count)
  Read raw 16-bit counter value from TPU channel.
- rx_err_t rx_tpu_encoder_read_count (rx_tpu_channel_t channel, rx_encoder_state_t *state)
  Read encoder count with automatic 16-bit overflow handling.
- rx_err_t rx_tpu_encoder_read_velocity (float *velocity_rps, float delta_time_s, rx_tpu_channel_t channel)
  Calculate encoder velocity in revolutions per second.
- rx_err_t rx_tpu_encoder_reset (rx_tpu_channel_t channel)
  Reset encoder count to zero.
- rx_err_t rx_tpu_encoder_set_count (int32_t count, rx_tpu_channel_t channel)
  Set encoder count to specific value.
- rx_err_t rx_tpu_encoder_deinit (rx_tpu_channel_t channel)
  Deinitialize TPU encoder.

5.1.1 Detailed Description

TPU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode).

Hardware-accelerated quadrature encoder interface using the RX72N Timer Pulse Unit (TPU) peripheral in phase counting mode. Provides zero-CPU-overhead encoder counting with automatic edge detection and direction sensing for the STAR robot's rear wheel encoders.

5.1.1.1 System Architecture Context

The STAR robot has 4 motors with 341 PPR Hall effect encoders. Front wheels use the MTU peripheral (see rx_mtu_encoder.h); rear wheels use the TPU peripheral handled by this module.

Encoder	Peripheral	Channel	Phase A Pin	Phase B Pin
0 (FL)	MTU1	MTU1	P24/MTCLKA (pin 33)	P25/MTCLKB (pin 32)
1 (FR)	MTU2	MTU2	PA1/MTCLKC (pin 96)	PC5/MTCLKD (pin 62)
2 (RL)	TPU1	TPU1	PC2/TCLKA (pin 70)	PA3/TCLKB (pin 94)
3 (RR)	TPU2	TPU2	PC0/TCLKC (pin 75)	PB3/TCLKD (pin 82)

5.1.1.2 Design Rationale

This module mirrors the rx_mtu_encoder.h API but operates on TPU channels. The same 16-bit overflow detection algorithm and velocity calculation logic are used, providing consistent behavior across front and rear encoders.

16-Bit Counter Overflow Handling:

```
delta = current_count - last_count
if (delta > 32768):   # Large positive jump
    delta -= 65536    # Counter underflowed
elif (delta < -32768):
    delta += 65536    # Counter overflowed
accumulated_count += delta
```

5.1.1.3 Performance Characteristics

Metric	Value	Notes
Counter update	Real-time	Hardware (zero CPU overhead)
Read latency	~0.5 us	Single register read
Overflow detection	~2 us	@ 240 MHz
Max safe poll interval (210 RPM)	6.8 s	32768 / (1364 * 3.5 RPS)
Actual poll rate	250 Hz	1666x safety margin

Module Dependencies:

Depends On:

- rx_err.h - Error codes
- rx_tpu.h - TPU hardware abstraction
- rx_mtu_encoder.h - Shared rx_encoder_state_t type

Used By:

- Motor control task - 250 Hz PID control loop (rear wheels)

NASA Power of 10 Compliance:

- Rule 1: No goto, setjmp, recursion
- Rule 2: All loops bounded
- Rule 3: No dynamic memory allocation
- Rule 4: All functions < 60 lines
- Rule 5: Minimum 2 validation checks per function
- Rule 6: Variables at smallest scope
- Rule 7: All return values checked
- Rule 8: C23 typed enums, minimal macros
- Rule 9: Single-level pointers only
- Rule 10: Compiles with -Wall -Wextra -Werror

See also

rx_mtu_encoder.h MTU encoder driver (front wheels)
rx_tpu.h TPU HAL driver
rx72n_tpu_regs.h TPU register definitions

Author

Locked, Inc.

Date

2026-02-10

Copyright

Since

Version 1.0.0

Definition in file rx_encoder_tpu.h.

5.1.2 Data Structure Documentation

5.1.2.1 struct rx_tpu_encoder_config_t

TPU encoder configuration structure.

Configuration parameters for initializing a quadrature encoder on a TPU channel. Mirrors rx_encoder_config_t but uses TPU channel identifiers.

STAR Project Configuration (back-wheel Hall harness is cross-wired, so

motor 2 = BR reads TPU2 and motor 3 = BL reads TPU1; see src/tasks/motor_control_task.c:2214-2216):

Motor	Channel	counts_per_rev	invert
2 (BR)	k_tpu_channel_2	1364	false
3 (BL)	k_tpu_channel_1	1364	true

Invariant

counts_per_rev > 0

channel must be 1, 2, 4, or 5

See also

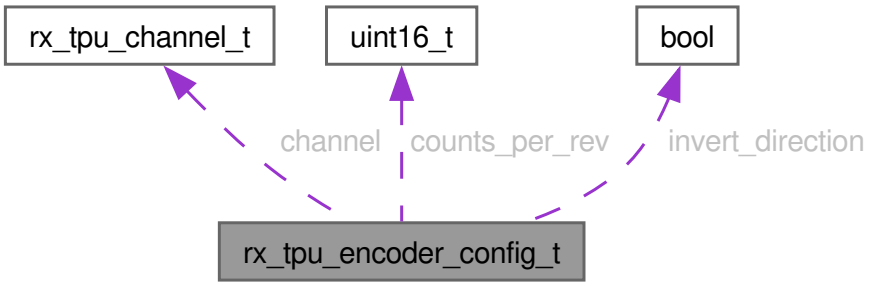
rx_tpu_encoder_init() Initialization function

Since

Version 1.0.0

Definition at line 123 of file rx_encoder_tpu.h.

Collaboration diagram for rx_tpu_encoder_config_t:



Data Fields

rx_tpu_channel_t	channel	TPU channel for encoder counting. Must be a phase-counting-capable channel (1, 2, 4, or 5).
uint16_t	counts_per_rev	Encoder counts per revolution (after 4x decoding). For STAR's 341 PPR encoders: 341 x 4 = 1364 counts/rev. Valid Range: 1 to 65535 Warning Zero causes division-by-zero in velocity calculation
bool	invert_direction	Invert encoder count direction. When true, reverses count direction to compensate for reversed wiring or motor mounting orientation. Software-only inversion.

5.1.3 Function Documentation

5.1.3.1 rx_tpu_encoder_deinit()

```
rx_err_t rx_tpu_encoder_deinit (
            const rx_tpu_channel_t channel)   [nodiscard]
```

Deinitialize TPU encoder.

Stops counter and releases resources. Can be reinitialized after.

Parameters

in	channel	TPU channel (1, 2, 4, or 5)

Returns

rx_err_t Error code

Return values

k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Precondition

Channel previously initialized

Postcondition

Counter stopped, channel marked uninitialized

See also

rx_tpu_encoder_init() Reinitialize after deinit

Since

Version 1.0.0

Deinitialize TPU encoder.

Calls rx_tpu_deinit() to stop the TPU counter and reset its mode, then clears the per-channel s_initialized flag and counts_per_rev. Subsequent reads on this channel will return k_rx_err_invalid_state until rx_tpu_encoder_init() is called again.

Note

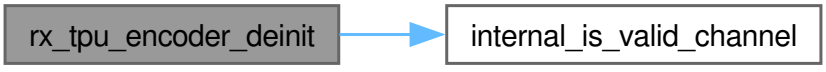
Not thread-safe with concurrent reads on the same channel. Typically called only at shutdown or before re-initialization.

Definition at line 479 of file rx_encoder_tpu.c.

```
00480 {
00481   if (!internal_is_valid_channel(channel)) {
00482     return k_rx_err_invalid_arg;
00483   }
00484
00485   if (!s_initialized[channel]) {
00486     rx_log_error(s_tag, "Deinit called on uninitialized channel");
00487     return k_rx_err_invalid_state;
00488   }
00489
00490   /* Deinitialize TPU HAL (stops counter, resets mode) */
00491   (void)rx_tpu_deinit(channel);
00492
00493   s_initialized[channel]    = false;
00494   s_counts_per_rev[channel] = 0;
00495
00496   rx_log_info(s_tag, "TPU encoder deinitialized");
00497
00498   return k_rx_ok;
00499 }
```

References internal_is_valid_channel(), s_counts_per_rev, s_initialized, and s_tag.

Here is the call graph for this function:



5.1.3.2 rx_tpu_encoder_init()

```
rx_err_t rx_tpu_encoder_init (
                const rx_tpu_encoder_config_t * config)   [nodiscard]
```

Initialize TPU channel for hardware quadrature encoder counting.

Configures a TPU channel in phase counting mode (4x decoding) and initializes software overflow tracking state. After initialization, the encoder is counting and ready for position/velocity reads.

Initialization steps:

1. Validate configuration parameters
2. Initialize TPU HAL in phase counting mode 4
3. Start the TPU counter
4. Initialize software overflow detection state

Parameters

in	config	Pointer to encoder configuration

Returns

rx_err_t Error code

Return values

k_rx_ok	Success, encoder initialized and counting
k_rx_err_null_ptr	config is nullptr
k_rx_err_invalid_arg	Invalid channel or counts_per_rev == 0
k_rx_err_invalid_state	Channel already initialized

Precondition

GPIO pins for TCLKA-D configured as peripheral I/O

Encoder physically connected

Postcondition

TPU channel in phase counting mode, counter running

Software state zeroed (count=0, position=0)

Note

Thread Safety: Not thread-safe, call during init only

See also

rx_tpu_encoder_deinit() Release resources

rx_tpu_encoder_read_count() Read position after init

Since

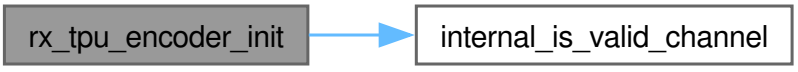
Version 1.0.0

Definition at line 209 of file rx_encoder_tpu.c.

```
00210 {
00211   RX_VALIDATE_PTR(config, s_tag, "config pointer is nullptr");
00212
00213   const rx_tpu_channel_t channel = config->channel;
00214
00215   if (!internal_is_valid_channel(channel)) {
00216     rx_log_error(s_tag, "Invalid TPU channel for encoder");
00217     return k_rx_err_invalid_arg;
00218   }
00219
00220   if (config->counts_per_rev < k_tpu_enc_min_counts_per_rev) {
00221     rx_log_error(s_tag, "counts_per_rev must be >= 1");
00222     return k_rx_err_invalid_arg;
00223   }
00224
00225   if (s_initialized[channel]) {
00226     rx_log_error(s_tag, "Channel already initialized");
00227     return k_rx_err_invalid_state;
00228   }
00229
00230   /* Configure TPU HAL for phase counting mode 4 (4x resolution) */
00231   const rx_tpu_config_t tpu_config = {
00232     .channel    = channel,
00233     .phase_mode = k_tpu_phase_mode_4,
00234   };
00235
00236   rx_err_t err = rx_tpu_init_phase_count(&tpu_config);
00237   if (err != k_rx_ok) {
00238     rx_log_error(s_tag, "TPU HAL init failed");
00239     return err;
00240   }
00241
00242   /* Start the counter (must succeed: channel was just initialized above) */
00243   (void)rx_tpu_start(channel);
00244
00245   /* Initialize software state */
00246   s_state[channel].total_count    = 0;
00247   s_state[channel].last_raw_count = 0;
00248   s_state[channel].revolutions    = 0;
00249   s_state[channel].position_deg   = k_tpu_enc_initial_position_deg;
00250
00251   s_counts_per_rev[channel]   = config->counts_per_rev;
00252   s_invert_direction[channel] = config->invert_direction;
00253   s_last_count[channel]       = 0;
00254   s_initialized[channel]      = true;
00255
00256   rx_log_info(s_tag, "TPU encoder initialized");
00257
00258   return k_rx_ok;
00259 }
```

References rx_tpu_encoder_config_t::channel, rx_tpu_encoder_config_t::counts_per_rev, internal_is_valid_channel(), rx_tpu_encoder_config_t::invert_direction, k_tpu_enc_initial_position_deg, k_tpu_enc_min_counts_per_rev, s_counts_per_rev, s_initialized, s_invert_direction, s_last_count, s_state, and s_tag.

Here is the call graph for this function:



5.1.3.3 rx_tpu_encoder_read_count()

```
rx_err_t rx_tpu_encoder_read_count (
                const rx_tpu_channel_t channel,
                rx_encoder_state_t * state)   [nodiscard]
```

Read encoder count with automatic 16-bit overflow handling.

Reads the current 16-bit hardware counter and updates the 32-bit accumulated count with automatic overflow/underflow correction. Also computes derived position (degrees) and revolution count.

Must be called frequently enough to catch overflows: at 210 RPM max motor speed with 1364 counts/rev, the safe interval is 6.8 seconds. The 250 Hz control loop provides a 1666x safety margin.

Parameters

in	channel	TPU channel (1, 2, 4, or 5)
out	state	Pointer to encoder state structure to update

Returns

rx_err_t Error code

Return values

k_rx_ok	Success, state updated
k_rx_err_null_ptr	state is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Precondition

Channel initialized via rx_tpu_encoder_init()

Called frequently enough to catch overflows

Postcondition

state->total_count updated with overflow correction

state->last_raw_count updated

state->revolutions and position_deg computed

Note

Thread Safety: Not thread-safe for same channel

See also

rx_tpu_encoder_read_velocity() Calculate velocity

Since

Version 1.0.0

Read encoder count with automatic 16-bit overflow handling.

Reads the raw TPU counter, then folds it through internal_update_state(), which extends the 16-bit hardware counter into a 32-bit signed total count by detecting wraparound between successive reads. Position in degrees and revolution count are derived from total_count and counts_per_rev.

Note

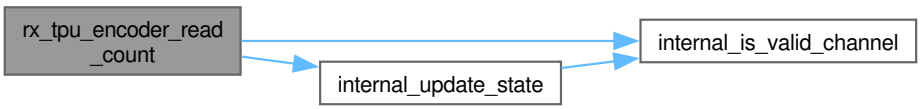
Not thread-safe with rx_tpu_encoder_reset() / _set_count() on the same channel because both update internal cached state.

Definition at line 302 of file rx_encoder_tpu.c.

```
00303 {
00304   RX_VALIDATE_PTR(state, s_tag, "state pointer is nullptr");
00305
00306   if (!internal_is_valid_channel(channel)) {
00307     return k_rx_err_invalid_arg;
00308   }
00309
00310   if (!s_initialized[channel]) {
00311     rx_log_error(s_tag, "Encoder not initialized");
00312     return k_rx_err_invalid_state;
00313   }
00314
00315   /* Read current 16-bit counter from TPU HAL (register read; must succeed) */
00316   uint16_t current_count = 0;
00317   (void)(rx_tpu_read_count(channel, &current_count));
00318
00319   return internal_update_state(state, channel, current_count);
00320 }
```

References internal_is_valid_channel(), internal_update_state(), s_initialized, and s_tag.

Here is the call graph for this function:



5.1.3.4 rx_tpu_encoder_read_raw()

```
rx_err_t rx_tpu_encoder_read_raw (
                const rx_tpu_channel_t channel,
                uint16_t * count)   [nodiscard]
```

Read raw 16-bit counter value from TPU channel.

Reads the current TCNT value without overflow handling. For position tracking with overflow correction, use rx_tpu_encoder_read_count().

Parameters

in	channel	TPU channel (1, 2, 4, or 5)

out	count	Pointer to store raw 16-bit count (0-65535)

Returns

rx_err_t Error code

Return values

k_rx_ok	Success
k_rx_err_null_ptr	count is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Precondition

Channel initialized via rx_tpu_encoder_init()

Postcondition

count contains current TCNT register value

Note

Thread Safety: Safe (single volatile register read)

See also

rx_tpu_encoder_read_count() Read with overflow handling

Since

Version 1.0.0

Read raw 16-bit counter value from TPU channel.

Bypasses overflow accumulation and returns the current TPU TCNT register directly via rx_tpu_read_count(). Useful for diagnostic dumps and low-level wraparound detection. Validates the channel index and the per-channel initialized flag before touching hardware.

Note

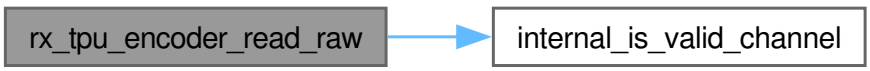
Not thread-safe with rx_tpu_encoder_reset() / _set_count() on the same channel. Concurrent reads on different channels are safe.

Definition at line 273 of file rx_encoder_tpu.c.

```
00274 {
00275   RX_VALIDATE_PTR(count, s_tag, "count pointer is nullptr");
00276
00277   if (!internal_is_valid_channel(channel)) {
00278     return k_rx_err_invalid_arg;
00279   }
00280
00281   if (!s_initialized[channel]) {
00282     rx_log_error(s_tag, "Encoder not initialized");
00283     return k_rx_err_invalid_state;
00284   }
00285
00286   return rx_tpu_read_count(channel, count);
00287 }
```

References internal_is_valid_channel(), s_initialized, and s_tag.

Here is the call graph for this function:



5.1.3.5 rx_tpu_encoder_read_velocity()

```
rx_err_t rx_tpu_encoder_read_velocity (
                float * velocity_rps,
                const float delta_time_s,
                const rx_tpu_channel_t channel)
```

Calculate encoder velocity in revolutions per second.

Derives angular velocity from the change in encoder position over a known time interval. Tracks previous count internally for delta calculation.

Parameters

out	velocity_rps	Pointer to store velocity (rev/sec)
in	delta_time_s	Time since last call in seconds (must be > 0)
in	channel	TPU channel (1, 2, 4, or 5)

Returns

rx_err_t Error code

Return values

k_rx_ok	Success

k_rx_err_null_ptr	velocity_rps is nullptr
k_rx_err_invalid_arg	Invalid delta_time_s or channel
k_rx_err_invalid_state	Encoder not initialized

Precondition

Channel initialized via rx_tpu_encoder_init()

delta_time_s matches actual elapsed time

Postcondition

velocity_rps contains calculated velocity

Internal last_count updated for next delta

Note

Parameter order: output, time, channel (prevents swaps)

Thread Safety: Not thread-safe for same channel

See also

rx_tpu_encoder_read_count() Read position

Since

Version 1.0.0

Calculate encoder velocity in revolutions per second.

Reads the latest cooked count, computes the delta against the cached s_last_count[channel] from the previous call, divides by counts_per_rev, and divides by delta_time_s to get rev/s. Updates the cached previous count so successive calls produce successive deltas. Logs a warning (but still returns k_rx_ok) when |velocity| exceeds k_tpu_enc_max_velocity_rps, indicating a likely encoder fault.

Note

Not thread-safe with itself or other encoder mutators on the same channel because of the shared s_last_count[] cache.

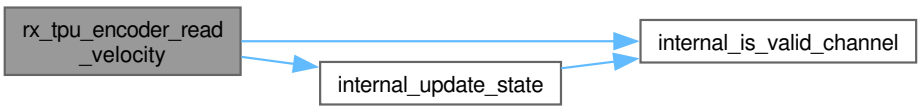
Definition at line 336 of file rx_encoder_tpu.c.

```
00339 {
00340   RX_VALIDATE_PTR(velocity_rps, s_tag, "velocity_rps pointer is nullptr");
00341
00342   const bool dt_too_small = (bool)(delta_time_s <= k_tpu_enc_min_delta_time_s);
00343   const bool dt_too_large = (bool)(delta_time_s > k_tpu_enc_max_delta_time_s);
00344   if ((bool)((int)dt_too_small | (int)dt_too_large)) {
00345     rx_log_error(s_tag, "Invalid delta time for velocity calculation");
00346     return k_rx_err_invalid_arg;
00347   }
00348
00349   if (!internal_is_valid_channel(channel)) {
00350     rx_log_error(s_tag, "Invalid channel");
00351     return k_rx_err_invalid_arg;
00352   }
```

```
00353
00354   if (!s_initialized[channel]) {
00355     rx_log_error(s_tag, "Encoder not initialized");
00356     return k_rx_err_invalid_state;
00357   }
00358
00359   /* Read current count via overflow handler (register read; must succeed) */
00360   uint16_t raw_count = 0;
00361   (void)rx_tpu_read_count(channel, &raw_count);
00362
00363   rx_encoder_state_t state;
00364   const rx_err_t     err = internal_update_state(&state, channel, raw_count);
00365   if (err != k_rx_ok) {
00366     return err;
00367   }
00368
00369   /* Calculate velocity from count delta */
00370   const int32_t delta_count = state.total_count - s_last_count[channel];
00371   s_last_count[channel]     = state.total_count;
00372
00373   /* cpr was validated by internal_update_state above; post-condition guard */
00374   const uint16_t cpr = s_counts_per_rev[channel];
00375
00376   const float delta_revs = (float)delta_count / (float)cpr;
00377   *velocity_rps          = delta_revs / delta_time_s;
00378
00379   /* Warn on unrealistic velocity */
00380   if (fabsf(*velocity_rps) > k_tpu_enc_max_velocity_rps) {
00381     rx_log_warn(s_tag, "Unrealistic velocity - possible encoder fault");
00382   }
00383
00384   return k_rx_ok;
00385 }
```

References internal_is_valid_channel(), internal_update_state(), k_tpu_enc_max_delta_time_s, k_tpu_enc_max_velocity_rps, k_tpu_enc_min_delta_time_s, s_counts_per_rev, s_initialized, s_last_count, s_tag, and rx_encoder_state_t::total_count.

Here is the call graph for this function:



5.1.3.6 rx_tpu_encoder_reset()

```
rx_err_t rx_tpu_encoder_reset (
                const rx_tpu_channel_t channel)   [nodiscard]
```

Reset encoder count to zero.

Resets both the TPU hardware counter and software accumulated state.

Parameters

in	channel	TPU channel (1, 2, 4, or 5)

Returns

rx_err_t Error code

Return values

k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Precondition

Channel initialized

Postcondition

TCNT=0, total_count=0, position=0

See also

rx_tpu_encoder_set_count() Set to specific value

Since

Version 1.0.0

Reset encoder count to zero.

Calls rx_tpu_reset_count() to clear the TPU TCNT register, then zeros the cached software state (total_count, last_raw_count, revolutions, position_deg, s_last_count). Used after carriage homing / index pulses or when re-initializing a faulted encoder.

Note

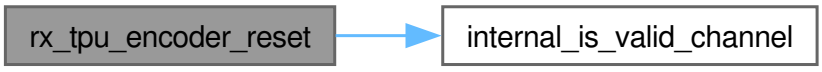
Not thread-safe with concurrent reads on the same channel.

Definition at line 398 of file rx_encoder_tpu.c.

```
00399 {
00400   if (!internal_is_valid_channel(channel)) {
00401     return k_rx_err_invalid_arg;
00402   }
00403
00404   if (!s_initialized[channel]) {
00405     rx_log_error(s_tag, "Encoder not initialized");
00406     return k_rx_err_invalid_state;
00407   }
00408
00409   /* Reset TPU hardware counter (register write; must succeed) */
00410   (void)(rx_tpu_reset_count(channel));
00411
00412   /* Reset software state */
00413   s_state[channel].total_count    = 0;
00414   s_state[channel].last_raw_count = 0;
00415   s_state[channel].revolutions    = 0;
00416   s_state[channel].position_deg   = k_tpu_enc_initial_position_deg;
00417   s_last_count[channel]           = 0;
00418
00419   return k_rx_ok;
00420 }
```

References internal_is_valid_channel(), k_tpu_enc_initial_position_deg, s_initialized, s_last_count, s_state, and s_tag.

Here is the call graph for this function:



5.1.3.7 rx_tpu_encoder_set_count()

```
rx_err_t rx_tpu_encoder_set_count (
                const int32_t count,
                const rx_tpu_channel_t channel)   [nodiscard]
```

Set encoder count to specific value.

Sets both hardware counter (low 16 bits) and software accumulated count.

Parameters

in	count	Count value to set
in	channel	TPU channel (1, 2, 4, or 5)

Returns

rx_err_t Error code

Return values

k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Precondition

Channel initialized

Postcondition

Software and hardware counts updated

Since

Version 1.0.0

Set encoder count to specific value.

Software-only counterpart to rx_tpu_encoder_reset(): does NOT touch the TPU TCNT register. Sets the cached total_count, last_raw_count, revolutions, and position_deg from the supplied count and the channel's counts_per_rev. Useful for tests, calibration, and resuming from a known offset. Detects a corrupted counts_per_rev (< minimum) and returns k_rx_err_invalid_state.

Note

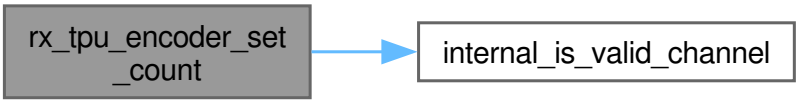
Not thread-safe with concurrent reads on the same channel.

Definition at line 435 of file rx_encoder_tpu.c.

```
00436 {
00437   if (!internal_is_valid_channel(channel)) {
00438     return k_rx_err_invalid_arg;
00439   }
00440
00441   if (!s_initialized[channel]) {
00442     rx_log_error(s_tag, "Encoder not initialized");
00443     return k_rx_err_invalid_state;
00444   }
00445
00446   /* Set software state */
00447   s_state[channel].total_count     = count;
00448   s_state[channel].last_raw_count = (uint16_t)((uint32_t)count & k_tpu_enc_16bit_mask);
00449   s_last_count[channel]           = count;
00450
00451   /* Recalculate position */
00452   const uint16_t cpr = s_counts_per_rev[channel];
00453   if (cpr < k_tpu_enc_min_counts_per_rev) {
00454     rx_log_error(s_tag, "counts_per_rev corrupted");
00455     return k_rx_err_invalid_state;
00456   }
00457
00458   s_state[channel].revolutions = count / cpr;
00459
00460   const int32_t remainder = count % cpr;
00461   s_state[channel].position_deg =
00462     ((float)remainder * (float)k_tpu_enc_degrees_per_rev) / (float)cpr;
00463
00464   return k_rx_ok;
00465 }
```

References internal_is_valid_channel(), k_tpu_enc_16bit_mask, k_tpu_enc_degrees_per_rev, k_tpu_enc_min_counts_per_rev, s_counts_per_rev, s_initialized, s_last_count, s_state, and s_tag.

Here is the call graph for this function:



5.2 rx_encoder_tpu.h

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_encoder_tpu.h
00003  * @brief TPU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode)
00004  *
00005  * @details
00006  * Hardware-accelerated quadrature encoder interface using the RX72N Timer Pulse
00007  * Unit (TPU) peripheral in phase counting mode. Provides zero-CPU-overhead
00008  * encoder counting with automatic edge detection and direction sensing for the
00009  * STAR robot's rear wheel encoders.
00010  *
00011  * ## System Architecture Context
00012  *
00013  * The STAR robot has 4 motors with 341 PPR Hall effect encoders. Front wheels
00014  * use the MTU peripheral (see rx_mtu_encoder.h); rear wheels use the TPU
00015  * peripheral handled by this module.
00016  *
00017  * | Encoder | Peripheral | Channel | Phase A Pin          | Phase B Pin          |
00018  * |---------|------------|---------|----------------------|----------------------|
00019  * | 0 (FL)  | MTU1       | MTU1    | P24/MTCLKA (pin 33) | P25/MTCLKB (pin 32) |
00020  * | 1 (FR)  | MTU2       | MTU2    | PA1/MTCLKC (pin 96) | PC5/MTCLKD (pin 62) |
00021  * | 2 (RL)  | TPU1       | TPU1    | PC2/TCLKA (pin 70)  | PA3/TCLKB (pin 94)  |
00022  * | 3 (RR)  | TPU2       | TPU2    | PC0/TCLKC (pin 75)  | PB3/TCLKD (pin 82)  |
00023  *
00024  * ## Design Rationale
```

```
00025  *
00026  * This module mirrors the rx_mtu_encoder.h API but operates on TPU channels.
00027  * The same 16-bit overflow detection algorithm and velocity calculation logic
00028  * are used, providing consistent behavior across front and rear encoders.
00029  *
00030  * **16-Bit Counter Overflow Handling:**
00031  * ```
00032  * delta = current_count - last_count
00033  * if (delta > 32768):   # Large positive jump
00034  *     delta -= 65536    # Counter underflowed
00035  * elif (delta < -32768):
00036  *     delta += 65536    # Counter overflowed
00037  * accumulated_count += delta
00038  * ```
00039  *
00040  * ## Performance Characteristics
00041  *
00042  * | Metric                        | Value       | Notes                        |
00043  * |-------------------------------|-------------|------------------------------|
00044  * | Counter update                | Real-time   | Hardware (zero CPU overhead)  |
00045  * | Read latency                  | ~0.5 us     | Single register read          |
00046  * | Overflow detection            | ~2 us       | @ 240 MHz                     |
00047  * | Max safe poll interval (210 RPM)| 6.8 s     | 32768 / (1364 * 3.5 RPS)      |
00048  * | Actual poll rate              | 250 Hz      | 1666x safety margin           |
00049  *
00050  * @par Module Dependencies:
00051  *
00052  * **Depends On:**
00053  * - [rx_err.h](../rx_core/inc/rx_err.h) - Error codes
00054  * - [rx_tpu.h](../rx_hal/inc/rx_tpu.h) - TPU hardware abstraction
00055  * - [rx_mtu_encoder.h](rx_mtu_encoder.h) - Shared rx_encoder_state_t type
00056  *
00057  * **Used By:**
00058  * - Motor control task - 250 Hz PID control loop (rear wheels)
00059  *
00060  * @par NASA Power of 10 Compliance:
00061  * - Rule 1: No goto, setjmp, recursion
00062  * - Rule 2: All loops bounded
00063  * - Rule 3: No dynamic memory allocation
00064  * - Rule 4: All functions < 60 lines
00065  * - Rule 5: Minimum 2 validation checks per function
00066  * - Rule 6: Variables at smallest scope
00067  * - Rule 7: All return values checked
00068  * - Rule 8: C23 typed enums, minimal macros
00069  * - Rule 9: Single-level pointers only
00070  * - Rule 10: Compiles with -Wall -Wextra -Werror
00071  *
00072  * @see rx_mtu_encoder.h MTU encoder driver (front wheels)
00073  * @see rx_tpu.h TPU HAL driver
00074  * @see rx72n_tpu_regs.h TPU register definitions
00075  *
00076  * @author Locked, Inc.
00077  * @date 2026-02-10
00078  * @copyright Copyright (c) 2026 Locked Inc.
00079  * SPDX-License-Identifier: MIT
00080  * @since Version 1.0.0
00081  */
00082
00083 #pragma once
00084
00085 #include <stdbool.h>
00086 #include <stdint.h>
00087
00088 #include "rx_err.h"
00089 #include "rx_mtu_encoder.h"
00090 #include "rx_tpu.h"
00091
00092 #ifdef __cplusplus
00093 extern "C" {
00094 #endif
00095
00096 /*
      ================================================================================
00097  * Type Definitions
00098  *
      ================================================================================
00099  */
00100
00101 /**
00102  * @struct rx_tpu_encoder_config_t
00103  * @brief TPU encoder configuration structure
00104  *
00105  * @details
00106  * Configuration parameters for initializing a quadrature encoder on a TPU
00107  * channel. Mirrors rx_encoder_config_t but uses TPU channel identifiers.
00108  *
00109  * @par STAR Project Configuration (back-wheel Hall harness is cross-wired, so
```

```
00110  *      motor 2 = BR reads TPU2 and motor 3 = BL reads TPU1; see
00111  *      src/tasks/motor_control_task.c:2214-2216):
00112  * | Motor   | Channel          | counts_per_rev | invert |
00113  * |---------|------------------|----------------|--------|
00114  * | 2 (BR)  | k_tpu_channel_2  | 1364           | false  |
00115  * | 3 (BL)  | k_tpu_channel_1  | 1364           | true   |
00116  *
00117  * @invariant counts_per_rev > 0
00118  * @invariant channel must be 1, 2, 4, or 5
00119  *
00120  * @see rx_tpu_encoder_init() Initialization function
00121  * @since Version 1.0.0
00122  */
00123 typedef struct {
00124     /**
00125      * @brief TPU channel for encoder counting
00126      * @details Must be a phase-counting-capable channel (1, 2, 4, or 5).
00127      */
00128     rx_tpu_channel_t channel;
00129
00130     /**
00131      * @brief Encoder counts per revolution (after 4x decoding)
00132      * @details For STAR's 341 PPR encoders: 341 x 4 = 1364 counts/rev.
00133      * @par Valid Range: 1 to 65535
00134      * @warning Zero causes division-by-zero in velocity calculation
00135      */
00136     uint16_t counts_per_rev;
00137
00138     /**
00139      * @brief Invert encoder count direction
00140      * @details When true, reverses count direction to compensate for reversed
00141      * wiring or motor mounting orientation. Software-only inversion.
00142      */
00143     bool invert_direction;
00144
00145 } rx_tpu_encoder_config_t;
00146
00147 /*
      ================================================================================
00148  * Public API
00149  *
      ================================================================================
00150  */
00151
00152 /**
00153  * @brief Initialize TPU channel for hardware quadrature encoder counting
00154  *
00155  * @details
00156  * Configures a TPU channel in phase counting mode (4x decoding) and
00157  * initializes software overflow tracking state. After initialization,
00158  * the encoder is counting and ready for position/velocity reads.
00159  *
00160  * Initialization steps:
00161  * 1. Validate configuration parameters
00162  * 2. Initialize TPU HAL in phase counting mode 4
00163  * 3. Start the TPU counter
00164  * 4. Initialize software overflow detection state
00165  *
00166  * @param[in] config Pointer to encoder configuration
00167  *
00168  * @return rx_err_t Error code
00169  * @retval k_rx_ok Success, encoder initialized and counting
00170  * @retval k_rx_err_null_ptr config is nullptr
00171  * @retval k_rx_err_invalid_arg Invalid channel or counts_per_rev == 0
00172  * @retval k_rx_err_invalid_state Channel already initialized
00173  *
00174  * @pre GPIO pins for TCLKA-D configured as peripheral I/O
00175  * @pre Encoder physically connected
00176  *
00177  * @post TPU channel in phase counting mode, counter running
00178  * @post Software state zeroed (count=0, position=0)
00179  *
00180  * @note Thread Safety: Not thread-safe, call during init only
00181  *
00182  * @see rx_tpu_encoder_deinit() Release resources
00183  * @see rx_tpu_encoder_read_count() Read position after init
00184  * @since Version 1.0.0
00185  */
00186 [[nodiscard]] rx_err_t rx_tpu_encoder_init(const rx_tpu_encoder_config_t* config);
00187
00188 /**
00189  * @brief Read raw 16-bit counter value from TPU channel
00190  *
00191  * @details
00192  * Reads the current TCNT value without overflow handling. For position
00193  * tracking with overflow correction, use rx_tpu_encoder_read_count().
00194  *
```

```
00195  * @param[in] channel TPU channel (1, 2, 4, or 5)
00196  * @param[out] count Pointer to store raw 16-bit count (0-65535)
00197  *
00198  * @return rx_err_t Error code
00199  * @retval k_rx_ok Success
00200  * @retval k_rx_err_null_ptr count is nullptr
00201  * @retval k_rx_err_invalid_arg Invalid channel
00202  * @retval k_rx_err_invalid_state Encoder not initialized
00203  *
00204  * @pre Channel initialized via rx_tpu_encoder_init()
00205  * @post count contains current TCNT register value
00206  *
00207  * @note Thread Safety: Safe (single volatile register read)
00208  *
00209  * @see rx_tpu_encoder_read_count() Read with overflow handling
00210  * @since Version 1.0.0
00211  */
00212 [[nodiscard]] rx_err_t rx_tpu_encoder_read_raw(rx_tpu_channel_t channel, uint16_t* count);
00213
00214 /**
00215  * @brief Read encoder count with automatic 16-bit overflow handling
00216  *
00217  * @details
00218  * Reads the current 16-bit hardware counter and updates the 32-bit
00219  * accumulated count with automatic overflow/underflow correction.
00220  * Also computes derived position (degrees) and revolution count.
00221  *
00222  * Must be called frequently enough to catch overflows: at 210 RPM max
00223  * motor speed with 1364 counts/rev, the safe interval is 6.8 seconds.
00224  * The 250 Hz control loop provides a 1666x safety margin.
00225  *
00226  * @param[in] channel TPU channel (1, 2, 4, or 5)
00227  * @param[out] state Pointer to encoder state structure to update
00228  *
00229  * @return rx_err_t Error code
00230  * @retval k_rx_ok Success, state updated
00231  * @retval k_rx_err_null_ptr state is nullptr
00232  * @retval k_rx_err_invalid_arg Invalid channel
00233  * @retval k_rx_err_invalid_state Encoder not initialized
00234  *
00235  * @pre Channel initialized via rx_tpu_encoder_init()
00236  * @pre Called frequently enough to catch overflows
00237  *
00238  * @post state->total_count updated with overflow correction
00239  * @post state->last_raw_count updated
00240  * @post state->revolutions and position_deg computed
00241  *
00242  * @note Thread Safety: Not thread-safe for same channel
00243  *
00244  * @see rx_tpu_encoder_read_velocity() Calculate velocity
00245  * @since Version 1.0.0
00246  */
00247 [[nodiscard]] rx_err_t rx_tpu_encoder_read_count(rx_tpu_channel_t    channel,
00248                                                  rx_encoder_state_t* state);
00249
00250 /**
00251  * @brief Calculate encoder velocity in revolutions per second
00252  *
00253  * @details
00254  * Derives angular velocity from the change in encoder position over a known
00255  * time interval. Tracks previous count internally for delta calculation.
00256  *
00257  * @param[out] velocity_rps Pointer to store velocity (rev/sec)
00258  * @param[in] delta_time_s Time since last call in seconds (must be > 0)
00259  * @param[in] channel TPU channel (1, 2, 4, or 5)
00260  *
00261  * @return rx_err_t Error code
00262  * @retval k_rx_ok Success
00263  * @retval k_rx_err_null_ptr velocity_rps is nullptr
00264  * @retval k_rx_err_invalid_arg Invalid delta_time_s or channel
00265  * @retval k_rx_err_invalid_state Encoder not initialized
00266  *
00267  * @pre Channel initialized via rx_tpu_encoder_init()
00268  * @pre delta_time_s matches actual elapsed time
00269  *
00270  * @post velocity_rps contains calculated velocity
00271  * @post Internal last_count updated for next delta
00272  *
00273  * @note Parameter order: output, time, channel (prevents swaps)
00274  * @note Thread Safety: Not thread-safe for same channel
00275  *
00276  * @see rx_tpu_encoder_read_count() Read position
00277  * @since Version 1.0.0
00278  */
00279 rx_err_t
00280 rx_tpu_encoder_read_velocity(float* velocity_rps, float delta_time_s, rx_tpu_channel_t channel);
00281
```

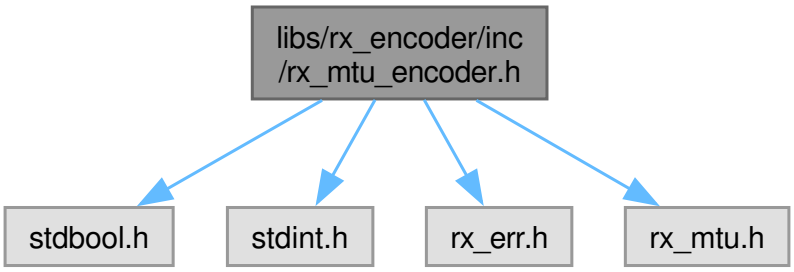
```
00282 /**
00283  * @brief Reset encoder count to zero
00284  *
00285  * @details
00286  * Resets both the TPU hardware counter and software accumulated state.
00287  *
00288  * @param[in] channel TPU channel (1, 2, 4, or 5)
00289  *
00290  * @return rx_err_t Error code
00291  * @retval k_rx_ok Success
00292  * @retval k_rx_err_invalid_arg Invalid channel
00293  * @retval k_rx_err_invalid_state Encoder not initialized
00294  *
00295  * @pre Channel initialized
00296  * @post TCNT=0, total_count=0, position=0
00297  *
00298  * @see rx_tpu_encoder_set_count() Set to specific value
00299  * @since Version 1.0.0
00300  */
00301 [[nodiscard]] rx_err_t rx_tpu_encoder_reset(rx_tpu_channel_t channel);
00302
00303 /**
00304  * @brief Set encoder count to specific value
00305  *
00306  * @details
00307  * Sets both hardware counter (low 16 bits) and software accumulated count.
00308  *
00309  * @param[in] count Count value to set
00310  * @param[in] channel TPU channel (1, 2, 4, or 5)
00311  *
00312  * @return rx_err_t Error code
00313  * @retval k_rx_ok Success
00314  * @retval k_rx_err_invalid_arg Invalid channel
00315  * @retval k_rx_err_invalid_state Encoder not initialized
00316  *
00317  * @pre Channel initialized
00318  * @post Software and hardware counts updated
00319  *
00320  * @since Version 1.0.0
00321  */
00322 [[nodiscard]] rx_err_t rx_tpu_encoder_set_count(int32_t count, rx_tpu_channel_t channel);
00323
00324 /**
00325  * @brief Deinitialize TPU encoder
00326  *
00327  * @details
00328  * Stops counter and releases resources. Can be reinitialized after.
00329  *
00330  * @param[in] channel TPU channel (1, 2, 4, or 5)
00331  *
00332  * @return rx_err_t Error code
00333  * @retval k_rx_ok Success
00334  * @retval k_rx_err_invalid_arg Invalid channel
00335  * @retval k_rx_err_invalid_state Encoder not initialized
00336  *
00337  * @pre Channel previously initialized
00338  * @post Counter stopped, channel marked uninitialized
00339  *
00340  * @see rx_tpu_encoder_init() Reinitialize after deinit
00341  * @since Version 1.0.0
00342  */
00343 [[nodiscard]] rx_err_t rx_tpu_encoder_deinit(rx_tpu_channel_t channel);
00344
00345 #ifdef UNIT_TEST
00346 /**
00347  * @brief Corrupt the counts_per_rev field for a channel (test-only)
00348  *
00349  * @details
00350  * Sets s_counts_per_rev[channel] = 0 to exercise runtime cpr-corrupted guards
00351  * in internal_update_state(), rx_tpu_encoder_read_velocity(), and
00352  * rx_tpu_encoder_set_count(). Available only in UNIT_TEST builds.
00353  *
00354  * @param[in] channel TPU channel (must be valid: 1, 2, 4, or 5)
00355  * @post s_counts_per_rev[channel] == 0
00356  *
00357  * @note Test-only; not compiled into production firmware
00358  * @since Version 1.0.0
00359  */
00360 void rx_tpu_encoder_test_corrupt_cpr(rx_tpu_channel_t channel);
00361 #endif /* UNIT_TEST */
00362
00363 #ifdef UNIT_TEST
00364 /**
00365  * @brief Direct access to internal_update_state for white-box testing (test-only)
00366  *
00367  * @details
00368  * Exposes the RX_STATIC_TESTABLE internal_update_state() function when compiled
```

```
00369  * with UNIT_TEST defined. Allows tests to exercise defensive guard paths that
00370  * are unreachable through the public API (null state, uninit channel, position
00371  * overflow) without modifying production code paths.
00372  *
00373  * @param[out] state Output state structure (may be NULL to test null guard)
00374  * @param[in] channel TPU channel (may be invalid to test channel guard)
00375  * @param[in] current_count Current 16-bit TCNT value
00376  *
00377  * @return rx_err_t Error code as returned by internal_update_state
00378  *
00379  * @since Version 1.0.0
00380  */
00381 rx_err_t
00382 internal_update_state(rx_encoder_state_t* state, rx_tpu_channel_t channel, uint16_t current_count);
00383 #endif /* UNIT_TEST */
00384
00385 #ifdef ___cplusplus
00386 }
00387 #endif
```

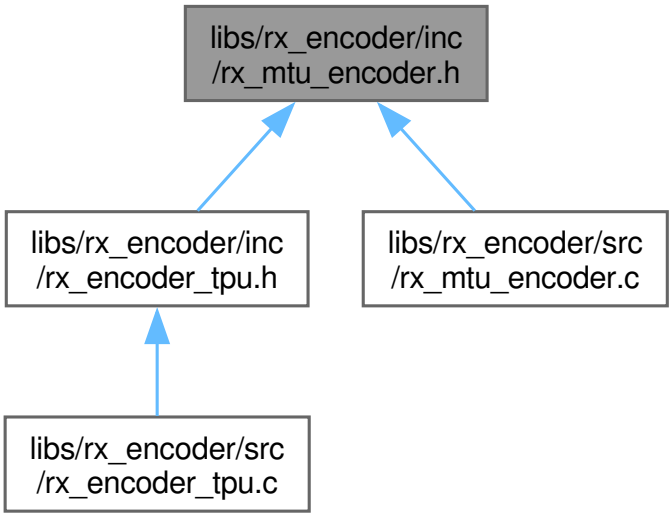
5.3 libs/rx_encoder/inc/rx_mtu_encoder.h File Reference

MTU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode).

#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
#include "rx_mtu.h"
Include dependency graph for rx_mtu_encoder.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct rx_encoder_config_t
  Encoder configuration structure.
- struct rx_encoder_state_t
  Encoder state tracking structure.

Functions

- rx_err_t rx_encoder_init (const rx_encoder_config_t *config)
  Initialize MTU channel for hardware quadrature encoder counting.
- rx_err_t rx_encoder_read_raw (rx_mtu_channel_t channel, uint16_t *count)
  Read raw encoder count.
- rx_err_t rx_encoder_read_count (rx_mtu_channel_t channel, rx_encoder_state_t *state)
  Read encoder count with automatic 16-bit overflow handling.
- rx_err_t rx_encoder_read_velocity (float *velocity_rps, float delta_time_s, rx_mtu_channel_t channel)
  Read encoder velocity.
- rx_err_t rx_encoder_reset (rx_mtu_channel_t channel)
  Reset encoder count to zero.
- rx_err_t rx_encoder_set_count (int32_t count, rx_mtu_channel_t channel)
  Set encoder count value.
- rx_err_t rx_encoder_deinit (rx_mtu_channel_t channel)
  Deinitialize encoder.

5.3.1 Detailed Description

MTU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode).

Hardware-accelerated quadrature encoder interface using the RX72N Multi-Function Timer (MTU) peripheral in phase counting mode. Provides zero-CPU-overhead encoder counting with automatic edge detection and direction sensing.

5.3.1.1 System Architecture Context

The STAR robot uses 4x 341 PPR (Pulses Per Revolution) Hall effect encoders for motor velocity feedback in a closed-loop PID control system. The MTU peripheral handles all encoder counting in hardware, freeing the CPU for control algorithm execution.

Signal Chain:

```
Motor Shaft -> Hall Encoder -> Phase A/B -> MTU (Phase Counting) -> CPU (Read Count)
     (341 PPR)      (Digital)    (4x Decode)          (Overflow Handling)
```

MTU Phase Counting Features:

- 4x decoding: Counts all edges of both Phase A and Phase B signals
- 16-bit counter: Range 0-65535, wraps automatically
- Hardware direction sensing: Increments/decrements based on A/B phase relationship
- Zero CPU overhead: No interrupts or polling required for counting
- Noise immunity: Built-in digital filter on input pins

5.3.1.2 Design Rationale

Why Hardware Encoder Counting:

- Performance: MTU counts at full bus speed (120 MHz), never misses edges
- Reliability: No software-induced count errors from interrupt latency
- Efficiency: Zero CPU cycles spent on edge detection
- Precision: 4x decoding provides 1364 counts/rev resolution

16-Bit Counter Limitation:

- Hardware constraint: MTU counter is 16-bit (0-65535)
- Solution: Software overflow detection via periodic polling
- Max speed before overflow: 48 revolutions (at 341 PPR x 4 = 1364 counts/rev)
- Required poll rate: >48 rev before overflow (safe at 250 Hz control loop)

Quadrature Encoding Fundamentals:

```
Phase A: ____/---\____/---\____
Phase B: ________/---\____/---\_

Forward:  A leads B (count up)
Backward: B leads A (count down)

4x Decoding: Count on rising + falling edges of both A and B
Result: 4 counts per encoder pulse (341 PPR x 4 = 1364 counts/rev)
```

5.3.1.3 Implementation Approach

Initialization:

1. Configure MTU channel pins (MTCLKA, MTCLKB) as inputs
2. Set MTU to phase counting mode (TGRA register)
3. Enable 4x decoding (counts all edges)
4. Initialize software state tracking
5. Start counter

Periodic Reading (250 Hz Control Loop):

1. Read 16-bit hardware counter value
2. Detect overflow/underflow since last read
3. Update 32-bit accumulated count
4. Calculate velocity from count delta
5. Feed velocity to PID controller

Overflow Detection Algorithm:

```
delta = current_count - last_count
if (delta > 32768):   # Large positive jump
    # Counter underflowed (wrapped 65535 -> 0)
    delta -= 65536
elif (delta < -32768): # Large negative jump
    # Counter overflowed (wrapped 0 -> 65535)
    delta += 65536
accumulated_count += delta
```

5.3.1.4 Performance Characteristics

Timing:

- Counter update: Hardware real-time (no software delay)
- Read latency: ~500ns (single TCNT register read)
- Overflow detection: ~2us @ 240 MHz

Maximum Speeds:

- Theoretical: 120 MHz / 4 edges = 30 MHz edge rate
- Encoder limit: 341 PPR x 4 x 10,000 RPM = 227 kHz (realistic max)
- Motor limit: 210 RPM = 1.2 kHz (actual operating speed)

Overflow Safety:

- 250 Hz polling: Can handle >12,000 RPM before missing overflow
- 100 Hz polling: Can handle >5,000 RPM before missing overflow
- Actual motor speed: 210 RPM (safe margin: 60x)

5.3.1.5 Memory Usage

Per Encoder:

- Configuration: 8 bytes (rx_encoder_config_t)
- State tracking: 16 bytes (rx_encoder_state_t)
- Hardware registers: 0 bytes RAM (memory-mapped I/O)
- Total: 24 bytes RAM per encoder

4 Encoders: 96 bytes total

Hardware Requirements:

Requirement	Value
CPU	Renesas RX72N
Peripheral	MTU (Multi-Function Timer) channels 1-2
Input Pins	MTCLKA/MTCLKB (MTU1), MTCLKC/MTCLKD (MTU2) (4 pins total)
Input Voltage	3.3V logic (Hall encoder output)
Counter Width	16-bit (0-65535)

Requirement	Value
Max Edge Rate	30 MHz (theoretical), 227 kHz (encoder limit)
Digital Filter	Hardware noise rejection

MTU Channel Allocation:

Motor	MTU Channel	Phase A Pin	Phase B Pin	Notes
Motor 0	MTU1	P24/MTCLKA (pin 33)	P25/MTCLKB (pin 32)	Front-left
Motor 1	MTU2	PA1/MTCLKC (pin 96)	PC5/MTCLKD (pin 62)	Front-right

Encoder Specifications (341 PPR Hall Effect):

Parameter	Value
Type	Hall effect (non-contact)
PPR (Pulses Per Revolution)	341
Channels	2 (Phase A, Phase B)
Output	Open collector (requires pull-up)
Supply Voltage	5V
Output Logic	3.3V compatible
Resolution (4x decode)	1364 counts/revolution
Angular Resolution	0.264deg per count
Max Speed	10,000 RPM (typical Hall encoder limit)

Module Dependencies:

Depends On:

- rx_err.h - Error codes
- rx_mtu.h - MTU hardware abstraction
- rx72n_regs.h - MTU register definitions

Used By:

- rx_motor.h - Motor velocity feedback
- Motor control task - 250 Hz PID control loop
- Odometry task - Robot position/velocity estimation

NASA Power of 10 Compliance:

Rule	Status	Notes
1. Simple control flow	[OK]	No goto, setjmp, recursion
2. Fixed loop bounds	[OK]	No unbounded loops
3. No dynamic memory	[OK]	Static allocation only
4. Function length <60 lines	[OK]	All functions <50 lines
5. Assertions	[OK]	2+ checks per function
6. Smallest scope	[OK]	Variables scoped minimally
7. Check return values	[OK]	All returns validated
8. Limited preprocessor	[OK]	Minimal macro usage
9. Pointer restrictions	[OK]	Single-level dereferencing
10. Compiler warnings	[OK]	-Wall -Wextra -Werror

SOLID Principles:

Single Responsibility (S):

- This module has one responsibility: quadrature encoder counting
- Separate concerns: rx_mtu handles hardware, this module handles encoder logic

Open/Closed (O):

- Configuration via rx_encoder_config_t (extensible)
- Can add new features (e.g., index pulse) without modifying existing API

Liskov Substitution (L):

- All MTU channels behave identically
- Mock implementations for testing

Interface Segregation (I):

- Focused API: init, read, reset, deinit
- Separate velocity calculation (optional)

Dependency Inversion (D):

- Depends on rx_mtu abstraction, not hardware registers
- Testable via HAL mocking

See also

rx_mtu.h MTU hardware abstraction layer
rx_motor.h Motor control using encoder feedback
docs/sections/03_hardware_pinout.tex Complete pin assignments
RX72N Hardware Manual Chapter 19 - Multi-Function Timer (MTU)

Author

Locked, Inc.

Date

2026-01-27

Copyright

Definition in file rx_mtu_encoder.h.

5.3.2 Data Structure Documentation

5.3.2.1 struct rx_encoder_config_t

Encoder configuration structure.

Configuration parameters for initializing a quadrature encoder on an MTU channel. This structure defines which MTU hardware channel to use, the encoder resolution, and direction inversion settings.

5.3.2.2 Configuration Guidelines

MTU Channel Selection:

- Use MTU1-MTU2 for encoder inputs (2 front motors)
- MTU1 uses MTCLKA/MTCLKB, MTU2 uses MTCLKC/MTCLKD
- Cannot share MTU channels between encoders

Counts Per Revolution:

- Calculate as: encoder_PPR x 4 (4x decoding)
- Example: 341 PPR encoder -> 1364 counts/rev
- Used for velocity and position calculations
- Must match physical encoder specification

Direction Inversion:

- Set true if motor rotation direction is opposite to desired
- Caused by encoder wiring (Phase A/B swapped) or mechanical mounting
- Software inversion (no hardware changes required)

Memory Layout:

Offset	Size	Field	Type	Notes
0	4	channel	rx_mtu_channel_t	Enum (uint32_t)
4	2	counts_per_rev	uint16_t	Max 65535
6	1	invert_direction	bool	Pad to 8 bytes
7	1	(padding)	-	Alignment
Total	8 bytes			

Invariant

counts_per_rev > 0 (zero counts invalid)

counts_per_rev <= 65536 (must fit in 16-bit counter)

channel must be valid MTU channel (MTU1-MTU2)

Basic Configuration Example:

```
#include "rx_mtu_encoder.h"

// Configure Motor 0 encoder (341 PPR, normal direction)
rx_encoder_config_t config = {
    .channel = k_rx_mtu_channel_1,          // MTU1
    .counts_per_rev = 341 * 4,              // 1364 counts/rev
    .invert_direction = false               // Normal direction
};

rx_err_t err = rx_encoder_init(&config);
if (err != k_rx_ok) {
    rx_log_error("ENCODER", "Initialization failed");
    return err;
}
```

Direction Inversion Example:

```
// Motor 1 has reversed encoder wiring - invert direction
rx_encoder_config_t motor1_config = {
    .channel = k_rx_mtu_channel_2,
    .counts_per_rev = 1364,
    .invert_direction = true  // Compensate for reversed wiring
};

rx_encoder_init(&motor1_config);
// Now positive counts mean forward motion (as expected)
```

All 4 Motors Configuration Example:

```
// STAR robot has 4 motors with 341 PPR encoders
const rx_encoder_config_t encoder_configs[4] = {
    {k_rx_mtu_channel_1, 1364, false},  // Motor 0: Front-left
    {k_rx_mtu_channel_2, 1364, true},   // Motor 1: Front-right (inverted)
    {k_rx_mtu_channel_3, 1364, false},  // Motor 2: Rear-left
    {k_rx_mtu_channel_4, 1364, false}   // Motor 3: Rear-right
};

// Initialize all encoders
for (uint8_t i = 0; i < 4; i++) {
    rx_err_t err = rx_encoder_init(&encoder_configs[i]);
    if (err != k_rx_ok) {
        rx_log_error("ENCODER", "Motor %d encoder init failed", i);
        return err;
    }
}
```

See also

rx_encoder_init() Initialize encoder with this configuration

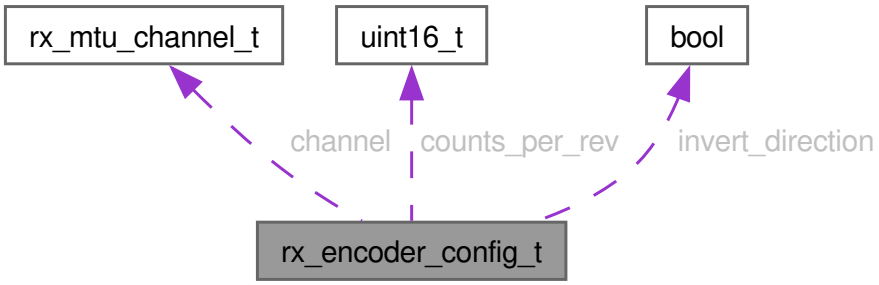
rx_mtu_channel_t MTU channel enumeration

Since

Version 1.0.0

Definition at line 318 of file rx_mtu_encoder.h.

Collaboration diagram for rx_encoder_config_t:



Data Fields

rx_mtu_channel_t	channel	MTU channel to use for encoder counting. Specifies which MTU peripheral channel (1-4) will be configured for phase counting mode. Each channel has dedicated MTCLKA and MTCLKB input pins that must be connected to the encoder's Phase A and Phase B outputs. Valid values: k_rx_mtu_channel_1 or k_rx_mtu_channel_2 Hardware Mapping (144-pin LFQFP): • MTU1: P24(MTCLKA, pin 33), P25(MTCLKB, pin 32) • MTU2: PA1(MTCLKC, pin 96), PC5(MTCLKD, pin 62) Note Validated by rx_encoder_init(), invalid channel returns k_rx_err_invalid_arg

uint16_t	counts_per_rev	Encoder counts per revolution (after 4x decoding). Total number of counts per complete shaft revolution when using 4x decoding. This is calculated as: encoder_PPR x 4 For STAR's 341 PPR Hall encoders: 341 x 4 = 1364 counts/revolution Used by velocity calculations to convert counts/second to revolutions/second. Value: 1364 (for 341 PPR encoders) Rationale: 341 PPR x 4x decoding = 1364 counts/rev Valid Range: 1 to 65535 Note Must match physical encoder specification Warning Zero value causes division-by-zero in velocity calculation
bool	invert_direction	Invert encoder count direction. When true, reverses the count direction (increments become decrements and vice versa). Used to compensate for: • Reversed encoder wiring (Phase A/B swapped) • Motor mounted in opposite orientation • Convention differences (clockwise vs counter-clockwise positive) Software inversion is applied during count reading, no hardware changes needed. Default: false (normal direction) Usage: Set true if motor direction is opposite to expected Note Does not affect hardware configuration, only software interpretation

5.3.2.3 struct rx_encoder_state_t

Encoder state tracking structure.

Runtime state maintained for each encoder to handle 16-bit hardware counter overflow and provide derived position/velocity values. This structure is updated by [rx_encoder_read_count()](rx_encoder_read_count) which implements overflow detection and accumulation.

5.3.2.4 Overflow Handling Strategy

The MTU hardware counter is 16-bit (0-65535), but encoders need to track unlimited rotations in both directions. This structure provides 32-bit signed accumulation via overflow detection:

Algorithm:

1. Read current 16-bit hardware count
2. Calculate delta from last_raw_count
3. Detect overflow: if (delta > 32768) -> underflow
4. Detect underflow: if (delta < -32768) -> overflow
5. Add corrected delta to total_count
6. Update last_raw_count for next iteration

Example:

```
last_raw_count = 65530
current_count = 10
delta = 10 - 65530 = -65520  (appears negative)
Since delta < -32768: overflow detected
corrected_delta = -65520 + 65536 = 16 (actual forward motion)
total_count += 16
```

5.3.2.5 Polling Requirements

Critical: Must call [rx_encoder_read_count()](rx_encoder_read_count) frequently enough to catch overflows before another wrap occurs.

Max Safe Interval:

- Encoder: 341 PPR x 4 = 1364 counts/rev
- Counter range: 65536 counts
- Safe margin: 32768 counts (half range)
- Max revolutions between reads: 32768 / 1364 = 24 revolutions

At 210 RPM (max motor speed):

- Revolution period: 285ms
- 24 revolutions: 6.8 seconds (safe time window)
- Required poll rate: >0.15 Hz (very conservative)
- Actual poll rate: 250 Hz (safe margin: 1666x)

Memory Layout:

Offset	Size	Field	Type	Notes
0	4	total_count	int32_t	Signed 32-bit
4	2	last_raw_count	uint16_t	Unsigned 16-bit
6	2	(padding)	-	Alignment
8	4	revolutions	int32_t	Signed 32-bit
12	4	position_deg	float	IEEE 754
Total	16 bytes			

Invariant

-2,147,483,648 <= total_count <= 2,147,483,647 (int32_t range)

0 <= last_raw_count <= 65535 (hardware counter range)

position_deg == (revolutions * 360.0) + fractional_degrees

revolutions == total_count / counts_per_rev

Basic Usage Example:

```
#include "rx_mtu_encoder.h"

// Initialize encoder
rx_encoder_config_t config = {
    .channel = k_rx_mtu_channel_1,
    .counts_per_rev = 1364,
    .invert_direction = false
};
rx_encoder_init(&config);

// Read encoder state (call at 250 Hz)
rx_encoder_state_t state;
rx_err_t err = rx_encoder_read_count(k_rx_mtu_channel_1, &state);
if (err == k_rx_ok) {
    printf("Position: %.2fdeg, Revolutions: %ld\n",
           state.position_deg, state.revolutions);
}
```

Velocity Calculation Example:

```
// Track encoder state across iterations
static rx_encoder_state_t last_state = {};
static bool first_iteration = true;

// Called at 250 Hz (every 4ms)
void control_loop_250hz(void)
{
    rx_encoder_state_t current_state;
    rx_encoder_read_count(k_rx_mtu_channel_1, &current_state);

    if (!first_iteration) {
        // Calculate velocity
        int32_t delta_counts = current_state.total_count - last_state.total_count;
        float dt_sec = 0.004f;  // 4ms = 0.004s
        float counts_per_sec = (float)delta_counts / dt_sec;
        float velocity_rps = counts_per_sec / 1364.0f;  // rev/sec
        float velocity_rpm = velocity_rps * 60.0f;      // rev/min

        printf("Velocity: %.2f RPM\n", velocity_rpm);
    }

    last_state = current_state;
    first_iteration = false;
}
```

Overflow Detection Example:

```
// Demonstrate overflow handling
rx_encoder_state_t state = {
    .total_count = 0,
    .last_raw_count = 65530,  // Near overflow
    .revolutions = 0,
    .position_deg = 0.0f
};

// First read: counter wraps 65530 -> 10
uint16_t current_count = 10;
int32_t delta = (int16_t)(current_count - state.last_raw_count);
// delta = -65520 (appears to go backward)
// But |delta| > 32768, so overflow detected
// Corrected: delta = -65520 + 65536 = 16 (forward)

state.total_count += delta;  // Now 16
state.last_raw_count = current_count;  // Now 10
// Correctly tracked forward motion across overflow
```

See also

rx_encoder_read_count() Function that updates this structure

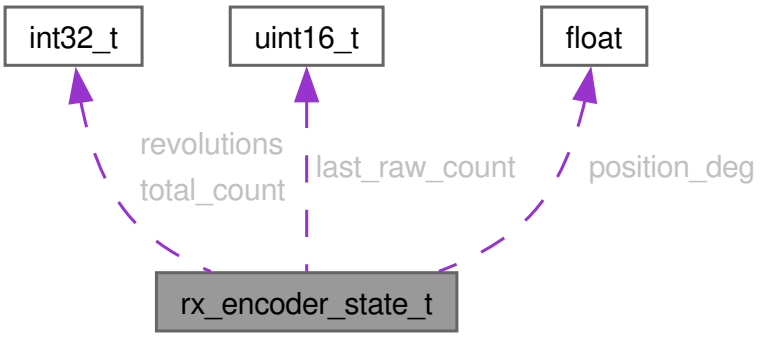
rx_encoder_read_velocity() Calculates velocity using state deltas

Since

Version 1.0.0

Definition at line 512 of file rx_mtu_encoder.h.

Collaboration diagram for rx_encoder_state_t:



Data Fields

uint16_t	last_raw_count	Last raw 16-bit hardware counter value (for overflow detection). Stores the previous hardware counter reading to enable delta calculation and overflow detection. This is the raw TCNT register value from the MTU peripheral (0-65535). Updated on every call to rx_encoder_read_count() after overflow correction. Units: Raw hardware counts Range: 0 to 65535 (16-bit unsigned) Note Do not modify manually; managed by rx_encoder_read_count() Warning Skipping reads can cause missed overflows if >32768 count change
float	position_deg	Angular position in degrees (0-360deg wrapping). Current shaft angle in degrees, calculated from total_count and counts_per_rev. Value wraps at 360deg (full revolution). Formula: position_deg = (total_count % counts_per_rev) * (360.0 / counts_per_rev) Provides high-resolution angular position (0.264deg per count at 1364 counts/rev). Units: Degrees Range: 0.0deg to 359.999deg Resolution: 0.264deg per count (360deg / 1364 counts) Wrapping: Value resets to 0deg after completing 360deg Note Use revolutions for multi-turn tracking, position_deg for angle within revolution

int32_t	revolutions	Number of complete revolutions (signed). Integer number of full shaft revolutions, calculated as: revolutions = total_count / counts_per_rev Positive values = forward revolutions, negative = reverse revolutions. Fractional revolutions are discarded (use position_deg for precision). Units: Complete revolutions Range: +/-1,575,331 revolutions (int32_t limit / 1364 counts/rev) Typical Values: -100 to +100 (normal operation) Calculation: revolutions = total_count / counts_per_rev
int32_t	total_count	Total accumulated count (signed 32-bit with overflow handling). Cumulative encoder count since initialization or last reset, accounting for all 16-bit hardware counter overflows/underflows. Positive values indicate forward rotation, negative values indicate reverse rotation. Range: +/-2.1 billion counts = +/-1.5 million revolutions (at 1364 counts/rev) This value is updated by rx_encoder_read_count() which detects and corrects for 16-bit counter wrap-around. Units: Encoder counts Range: -2,147,483,648 to +2,147,483,647 (int32_t limits) Typical Values: -10,000 to +10,000 (few revolutions) Note Overflow of this 32-bit value would require ~1.5M revolutions (unlikely)

5.3.3 Function Documentation

5.3.3.1 rx_encoder_deinit()

```
rx_err_t rx_encoder_deinit (
            const rx_mtu_channel_t channel)   [nodiscard]
```

Deinitialize encoder.

Stops counter and releases resources.

Parameters

in	channel	MTU channel

Returns

k_rx_ok on success

k_rx_err_invalid_arg if channel is invalid

Deinitialize encoder.

Performs orderly shutdown of encoder by stopping MTU timer and clearing initialization flag. After deinitialization, encoder can be reinitialized with different configuration parameters via rx_encoder_init().

Deinitialization Sequence:

1. Validate channel is valid and initialized
2. Stop MTU timer (halt edge counting)
3. Clear initialized flag (mark channel as uninitialized)
4. Clear counts_per_rev (prevent accidental division by stale value)

Use Cases:

- System shutdown: Release resources before power-down
- Reconfiguration: Change encoder parameters (requires deinit -> init)
- Error recovery: Clean up after hardware fault
- Resource sharing: Free MTU channel for other use

Precondition

channel must be initialized via rx_encoder_init()

No other tasks are currently accessing this encoder

Postcondition

s_encoder_initialized[channel] = false

MTU timer stopped (no longer counting)

s_counts_per_rev[channel] = 0

Other state (total_count, position) preserved but inaccessible

Note

If rx_mtu_stop() fails, warning logged but deinit continues

Timer stop affects entire channel (shared resource)

Thread-safe only with external synchronization

Warning

Ensure no other tasks are using encoder before calling

Attention

Encoder state preserved but inaccessible until reinitialized

Performance:

- Execution time: ~2 us @ 240 MHz (timer stop + flag clear)
- Called once per encoder during shutdown (not performance-critical)

Example - Clean Shutdown:

```
// Shutdown encoder before system power-down
rx_encoder_state_t final_state;
rx_encoder_read_count(k_mtu_channel_1, &final_state);
rx_log_info("SHUTDOWN", "Final position: %d counts", final_state.total_count);

rx_err_t err = rx_encoder_deinit(k_mtu_channel_1);
if (err != k_rx_ok) {
    rx_log_error("SHUTDOWN", "Failed to deinit encoder: %d", err);
}
// Encoder no longer usable
```

Example - Reconfiguration:

```
// Change encoder from 1364 CPR to 2048 CPR
rx_encoder_deinit(k_mtu_channel_1);

rx_encoder_config_t new_config = {
    .channel = k_mtu_channel_1,
    .counts_per_rev = 2048,  // Changed from 1364
    .invert_direction = false,
};
rx_encoder_init(&new_config);
```

See also

rx_encoder_init() Reinitialize after deinit

rx_mtu_stop() Underlying timer stop function

Since

Version 1.0.0

NASA Power of 10 Compliance:

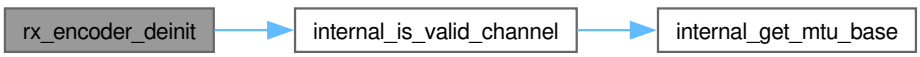
- Rule 1: [OK] No goto or recursion
- Rule 5: [OK] 2 preconditions (channel valid, initialized)
- Rule 5: [OK] 1 postcondition (initialized flag cleared)

Definition at line 1163 of file rx_mtu_encoder.c.

```
01164 {
01165   if (!internal_is_valid_channel(channel)) {
01166     return k_rx_err_invalid_arg;
01167   }
01168
01169   if (!s_encoder_initialized[channel]) {
01170     rx_log_error(s_tag, "Deinit called on uninitialized channel");
01171     return k_rx_err_invalid_state;
01172   }
01173
01174   /* Stop timer (explicitly ignore return value on cleanup path) */
01175   (void)rx_mtu_stop(channel);
01176
01177   /* Mark as uninitialized */
01178   s_encoder_initialized[channel] = false;
01179   s_counts_per_rev[channel]      = k_encoder_count_reset;
01180
01181   rx_log_info(s_tag, "MTU encoder deinitialized");
01182
01183   return k_rx_ok;
01184 }
```

References internal_is_valid_channel(), k_encoder_count_reset, s_counts_per_rev, s_encoder_initialized, and s_tag.

Here is the call graph for this function:



5.3.3.2 rx_encoder_init()

```
rx_err_t rx_encoder_init (
                const rx_encoder_config_t * config)   [nodiscard]
```

Initialize MTU channel for hardware quadrature encoder counting.

Configures a Multi-Function Timer (MTU) channel in phase counting mode for hardware-accelerated quadrature encoder decoding. This function:

1. Validates configuration parameters
2. Configures MTU pins (MTCLKA, MTCLKB) as inputs
3. Sets MTU mode to phase counting with 4x decoding
4. Initializes software state tracking (zero position)
5. Starts hardware counter

5.3.3.3 Algorithm Steps

1. Validate Parameters:
   - Check config pointer is not nullptr
   - Verify MTU channel is valid (MTU1-MTU2)
   - Ensure counts_per_rev > 0 and <= 65536
2. Configure MTU Pins:
   - Set MTCLKA, MTCLKB pins as digital inputs

- Enable pull-up resistors (for open-collector encoders)
- Configure pin function select registers (PFS)

3. Configure MTU Mode:
   - Set phase counting mode (TGRA control)
   - Enable 4x decoding (count all A/B edges)
   - Configure counter direction based on phase relationship
   - Apply direction inversion if requested
4. Initialize State:
   - Zero hardware counter (TCNT = 0)
   - Initialize software state structure
   - Clear any pending overflow flags
5. Start Counter:
   - Enable MTU channel
   - Counter now tracks encoder in hardware

5.3.3.4 Performance Analysis

Execution Time:

- Best case: ~15us @ 240 MHz (valid config, hardware ready)
- Worst case: ~20us @ 240 MHz (with error checking)
- Average case: ~17us @ 240 MHz

Memory Usage:

- Stack: ~48 bytes (local variables + function call overhead)
- Static RAM: 24 bytes per encoder (config + state)
- No heap allocation

Parameters

in	config	Pointer to encoder configuration structure • Must not be nullptr • All fields must be valid • Configuration is copied; pointer need not remain valid after init

Returns

rx_err_t Error code indicating success or failure

Return values

k_rx_ok	Success, encoder initialized and counting
k_rx_err_null_ptr	config pointer is nullptr
k_rx_err_invalid_arg	Invalid MTU channel (not MTU1-MTU2); counts_per_rev is 0 (division by zero); counts_per_rev > 65536 (counter overflow)
k_rx_err_invalid_state	MTU peripheral not clocked (MSTPCR disabled)
k_rx_err_hardware	MTU configuration failed (hardware fault)

Precondition

MTU peripheral clock must be enabled (MSTPCRA.MSTPA9 = 0)

GPIO pins for MTCLKA/MTCLKB must be available (not used by other peripherals)

Encoder must be connected and powered

System must be in run mode (not low-power sleep)

Postcondition

MTU channel configured in phase counting mode

Hardware counter initialized to zero

Software state structure initialized

Encoder counting started automatically

Subsequent rx_encoder_read_count() calls will return valid data

Invariant

Channel remains in phase counting mode until rx_encoder_deinit()

Counter increments on forward rotation, decrements on reverse

4x decoding active (all A/B edges counted)

Note

Thread Safety: Not thread-safe. Call from single thread or protect with mutex.

Re-entrancy: Not reentrant. Do not call from interrupt context.

Performance: One-time initialization, ~17us execution time

Memory: Allocates 24 bytes static RAM per encoder

Warning

Call this function ONCE per encoder during system initialization

Do not call while encoder is in use (stop motor first)

Calling twice on same channel without deinit causes undefined behavior

Attention

Encoder must be physically connected before initialization

counts_per_rev must match physical encoder specification exactly

Basic Initialization Example:

```
#include "rx_mtu_encoder.h"
#include "rx_log.h"

// Initialize Motor 0 encoder (341 PPR Hall effect)
void init_motor0_encoder(void)
{
    rx_encoder_config_t config = {
        .channel = k_rx_mtu_channel_1,    // MTU1
        .counts_per_rev = 1364,           // 341 PPR x 4
        .invert_direction = false         // Normal wiring
    };

    rx_err_t err = rx_encoder_init(&config);
    if (err != k_rx_ok) {
        rx_log_error("ENCODER", "Motor 0 init failed: %d", err);
        return;
    }

    rx_log_info("ENCODER", "Motor 0 encoder ready");
}
```

Error Handling Example:

```
rx_encoder_config_t config = {};

rx_err_t err = rx_encoder_init(&config);
switch (err) {
    case k_rx_ok:
        // Success - encoder ready
        break;

    case k_rx_err_null_ptr:
        rx_log_error("ENCODER", "nullptr config pointer");
        return k_rx_err_invalid_arg;

    case k_rx_err_invalid_arg:
        rx_log_error("ENCODER", "Invalid channel or counts_per_rev");
        return k_rx_err_invalid_arg;

    case k_rx_err_invalid_state:
        rx_log_error("ENCODER", "MTU peripheral not clocked");
        // Enable MTU clock: SYSTEM.MSTPCRA &= ~(1 << 9);
        return k_rx_err_invalid_state;

    default:
        rx_log_error("ENCODER", "Unexpected error: %d", err);
        return err;
}
```

All Motors Initialization Example:

```
// Initialize all 4 motor encoders with error checking
rx_err_t init_all_encoders(void)
{
    const rx_encoder_config_t configs[4] = {
        {k_rx_mtu_channel_1, 1364, false},  // Motor 0
        {k_rx_mtu_channel_2, 1364, true},   // Motor 1 (inverted)
        {k_rx_mtu_channel_3, 1364, false},  // Motor 2
        {k_rx_mtu_channel_4, 1364, false}   // Motor 3
    };

    for (uint8_t i = 0; i < 4; i++) {
        rx_err_t err = rx_encoder_init(&configs[i]);
        if (err != k_rx_ok) {
            rx_log_error("ENCODER", "Motor %d init failed: %d", i, err);
            // Clean up already-initialized encoders
            for (uint8_t j = 0; j < i; j++) {
                rx_encoder_deinit(configs[j].channel);
            }
```

```
            return err;
        }
    }

    rx_log_info("ENCODER", "All 4 encoders initialized");
    return k_rx_ok;
}
```

Custom Encoder Configuration Example:

```
// High-resolution encoder: 1024 PPR x 4 = 4096 counts/rev
rx_encoder_config_t high_res_config = {
    .channel = k_rx_mtu_channel_1,
    .counts_per_rev = 4096,
    .invert_direction = false
};

// Low-resolution encoder: 64 PPR x 4 = 256 counts/rev
rx_encoder_config_t low_res_config = {
    .channel = k_rx_mtu_channel_2,
    .counts_per_rev = 256,
    .invert_direction = false
};
```

See also

rx_encoder_deinit() Cleanup encoder when no longer needed

rx_encoder_read_count() Read encoder position after initialization

rx_encoder_reset() Zero encoder position without deinitializing

rx_mtu.h MTU hardware abstraction layer

docs/sections/03_hardware_pinout.tex MTU pin assignments

Since

Version 1.0.0

Version

1.0.0

Test test_rx_mtu_encoder.c::test_encoder_init() Unit test coverage

NASA Power of 10 Compliance:

- Rule 5: 6 preconditions, 5 postconditions [OK]
- Rule 7: All return values documented and validated by caller [OK]

Initialize MTU channel for hardware quadrature encoder counting.

Configures specified MTU channel in phase counting mode (4x decoding) for hardware-accelerated quadrature encoder counting. This function performs complete initialization including module enable, timer configuration, state initialization, and verification.

Initialization Sequence:

1. Validate configuration parameters (NULL check, counts_per_rev range)

2. Enable MTU peripheral module (clear MSTPCRA module stop bit)
3. Stop timer before configuration (prevents glitches during setup)
4. Configure MTU registers (TMDR=0x04 for phase counting, TCR for external clock)
5. Initialize software state (total_count, position, velocity tracking)
6. Start timer (begin counting encoder edges)
7. Verify timer is counting (post-condition check per NASA Rule 5)

Hardware Configuration Applied:

- TMDR (Timer Mode Register) = 0x04 -> Phase counting mode 1 (4x decoding)
- TCR (Timer Control Register) = 0x00 -> External clock, no prescaler
- TIOR (I/O Control) = 0x00 -> Disabled (not used in phase counting mode)
- TCNT (Counter Register) = 0 -> Reset to zero

Encoder Resolution Calculation: For STAR project motors (341 PPR encoder):

- Manufacturer PPR: 341 pulses/revolution
- 4x decoding: 341 x 4 = 1364 counts/revolution
- Angular resolution: 360deg / 1364 = 0.264deg per count

Precondition

config must point to valid rx_encoder_config_t structure
MTU channel must not already be in use (no double initialization check)
GPIO pins (MTCLKA/MTCLKB) must be configured for peripheral function
Encoder must be physically connected to MTCLKA/MTCLKB pins

Postcondition

On success: s_encoder_initialized[channel] = true
On success: MTU timer counting encoder edges
On success: Software state initialized (count=0, position=0deg, rev=0)
On failure: s_encoder_initialized[channel] = false, timer stopped

Note

Thread-safe only if different channels initialized from different tasks
GPIO pin configuration must be done externally (not handled by this function)
Counter starts at zero regardless of actual encoder position
Can only initialize once per channel (no re-initialization without deinit)

Warning

Must configure GPIO pins before calling this function

Encoder motion during initialization may cause initial count offset

Attention

Timer mode verification reads back hardware registers (post-condition check)

Performance:

- Execution time: ~25 us @ 240 MHz (includes module enable and register verification)
- One-time initialization cost, called once per encoder at system startup

Example - Single Encoder:

```
// Configure encoder for motor 0 (front-left wheel)
rx_encoder_config_t encoder_config = {
    .channel = k_mtu_channel_1,
    .counts_per_rev = 1364,      // 341 PPR x 4 (4x decoding)
    .invert_direction = false,   // Normal counting direction
};

rx_err_t err = rx_encoder_init(&encoder_config);
if (err != k_rx_ok) {
    rx_log_error("APP", "Failed to initialize encoder: %d", err);
    return err;
}

// Encoder now counting, can read position/velocity
```

Example - Four Encoders (STAR Platform):

```
// STAR robot: 4 motors with 341 PPR encoders
const rx_encoder_config_t encoder_configs[4] = {
    { .channel = k_mtu_channel_1, .counts_per_rev = 1364, .invert_direction = false },
    { .channel = k_mtu_channel_2, .counts_per_rev = 1364, .invert_direction = true },
    { .channel = k_mtu_channel_3, .counts_per_rev = 1364, .invert_direction = false },
    { .channel = k_mtu_channel_4, .counts_per_rev = 1364, .invert_direction = true },
};

for (uint8_t i = 0; i < 4; i++) {
    rx_err_t err = rx_encoder_init(&encoder_configs[i]);
    if (err != k_rx_ok) {
        // Cleanup previously initialized encoders
        for (uint8_t j = 0; j < i; j++) {
            (void)rx_encoder_deinit(encoder_configs[j].channel);
        }
        return err;
    }
}
rx_log_info("APP", "All 4 encoders initialized");
```

See also

rx_encoder_deinit() Cleanup function

rx_encoder_read_count() Read position after initialization

rx_encoder_read_velocity() Read velocity after initialization

rx_encoder_config_t Configuration structure definition

Since

Version 1.0.0

NASA Power of 10 Compliance:

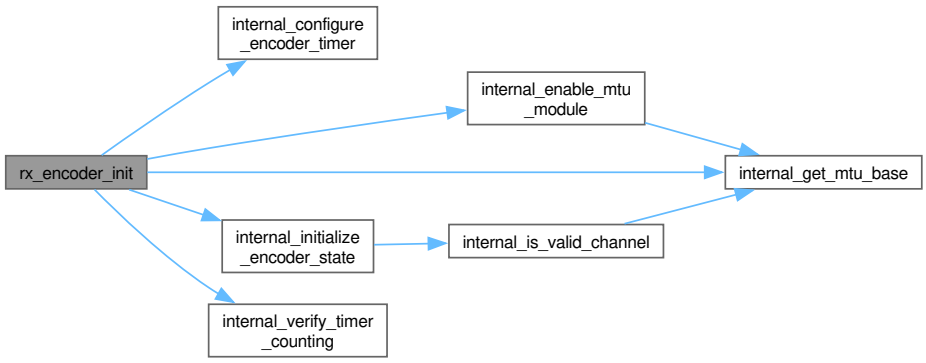
- Rule 1: [OK] No goto or recursion, sequential initialization steps
- Rule 3: [OK] No dynamic memory allocation (static state arrays)
- Rule 5: [OK] 3 preconditions (NULL check, range check, channel valid)
- Rule 5: [OK] 2 postconditions (initialized flag set, timer counting)
- Rule 7: [OK] All return values checked (enable, stop, configure, start, verify)

Definition at line 607 of file rx_mtu_encoder.c.

```
00608 {
00609   /* Validate inputs */
00610   RX_VALIDATE_PTR(config, s_tag, "config pointer is nullptr");
00611
00612   const rx_mtu_channel_t channel = config->channel;
00613
00614   /* Upper bound check omitted - counts_per_rev is uint16_t, k_encoder_max_counts_per_rev == 65535
00615    * (uint16_t max), so counts_per_rev > k_encoder_max_counts_per_rev is always false (-Wtype-limits) */
00616   if (config->counts_per_rev < k_encoder_min_counts_per_rev) {
00617     rx_log_error(s_tag, "counts_per_rev must be >= 1");
00618     return k_rx_err_invalid_arg;
00619   }
00620
00621   volatile rx_mtu_channel_regs_t* const mtu = internal_get_mtu_base(channel);
00622   if (mtu == nullptr) {
00623     return k_rx_err_invalid_arg;
00624   }
00625
00626   /* Enable MTU module */
00627   (void)(internal_enable_mtu_module(channel));
00628   /* channel already validated by internal_get_mtu_base above - cannot fail */
00629
00630   /* Stop timer before configuration */
00631   (void)(rx_mtu_stop(channel));
00632   /* channel already validated - rx_mtu_stop cannot fail for valid channels */
00633
00634   /* Configure timer for encoder mode (mtu is non-null, checked above) */
00635   (void)(internal_configure_encoder_timer(mtu));
00636   /* mtu is already validated non-null - configure cannot fail */
00637
00638   /* Initialize state */
00639   (void)(internal_initialize_encoder_state(channel, config));
00640   /* channel and config already validated above - cannot fail */
00641
00642   /* Start counter */
00643   (void)(rx_mtu_start(channel));
00644   /* channel already validated - rx_mtu_start cannot fail for valid channels */
00645
00646   /* Post-condition: Verify timer is counting.
00647    * configure_encoder_timer already set tmdr = k_tmdr_phase_counting_mode_1
00648    * and mtu is non-null, so this cannot fail in normal operation. */
00649   (void)(internal_verify_timer_counting(mtu));
00650
00651   rx_log_info(s_tag, "MTU encoder initialized successfully");
00652   return k_rx_ok;
00653 }
```

References rx_encoder_config_t::channel, rx_encoder_config_t::counts_per_rev, internal_configure_encoder_timer(), internal_enable_mtu_module(), internal_get_mtu_base(), internal_initialize_encoder_state(), internal_verify_timer_counting(), k_encoder_min_counts_per_rev, and s_tag.

Here is the call graph for this function:



5.3.3.5 rx_encoder_read_count()

```
rx_err_t rx_encoder_read_count (
            const rx_mtu_channel_t channel,
            rx_encoder_state_t * state)    [nodiscard]
```

Read encoder count with automatic 16-bit overflow handling.

Reads the current 16-bit hardware counter value and updates the accumulated 32-bit signed count, automatically detecting and correcting for counter overflow/underflow. This is the primary function for encoder position tracking.

5.3.3.6 Algorithm Steps

1. Read Hardware Counter:
   - Read MTU TCNT register (16-bit value 0-65535)
2. Calculate Delta:
   - delta = current_count - last_raw_count
   - Cast to int16_t for signed arithmetic
3. Detect Overflow/Underflow:
   - If delta > 32768: Counter underflowed (wrapped 65535 -> 0)
     - Correct: delta -= 65536
   - If delta < -32768: Counter overflowed (wrapped 0 -> 65535)
     - Correct: delta += 65536
4. Update Accumulated Count:
   - total_count += delta
5. Calculate Derived Values:
   - revolutions = total_count / counts_per_rev
   - position_deg = (total_count % counts_per_rev) * (360.0 / counts_per_rev)
6. Store Current Count:
   - last_raw_count = current_count

5.3.3.7 Overflow Detection Example

Forward motion across overflow:

```
last_raw_count = 65530
current_count = 10
delta = 10 - 65530 = -65520 (appears to go backward!)
|delta| = 65520 > 32768 -> overflow detected
corrected_delta = -65520 + 65536 = 16 (actual forward motion)
```

Backward motion across underflow:

```
last_raw_count = 10
current_count = 65530
delta = 65530 - 10 = 65520 (appears to jump forward!)
delta = 65520 > 32768 -> underflow detected
corrected_delta = 65520 - 65536 = -16 (actual backward motion)
```

5.3.3.8 Polling Requirements

CRITICAL: This function MUST be called frequently enough to catch overflows before another wrap occurs (within 32768 counts).

Safe Polling Rate:

- Max change between reads: 32768 counts (half of 16-bit range)
- At 1364 counts/rev: 32768 / 1364 = 24 revolutions
- At 210 RPM max speed: 24 rev / 3.5 rev/sec = 6.8 seconds
- Minimum poll rate: 0.15 Hz (once per 6.8 seconds)
- Recommended poll rate: 250 Hz (1666x safety margin)

5.3.3.9 Performance Analysis

Execution Time:

- Best case: ~2us @ 240 MHz (no overflow)
- Worst case: ~3us @ 240 MHz (with overflow correction)
- Average case: ~2.5us @ 240 MHz

Memory Usage:

- Stack: ~32 bytes (local variables)
- No heap allocation

Parameters

in	channel	MTU channel to read (MTU1-MTU2) • Must be previously initialized via rx_encoder_init() • Invalid channel returns k_rx_err_invalid_arg
out	state	Pointer to encoder state structure to update • Must not be nullptr • All fields updated on success • Unchanged on error

Returns

rx_err_t Error code indicating success or failure

Return values

k_rx_ok	Success, state updated with current encoder position
k_rx_err_null_ptr	state pointer is nullptr
k_rx_err_invalid_arg	Invalid MTU channel (not MTU1-MTU2)
k_rx_err_invalid_state	Encoder not initialized (call rx_encoder_init first)
k_rx_err_hardware	Hardware read failed (MTU peripheral fault)

Precondition

Encoder initialized via rx_encoder_init()
MTU peripheral clocked and running
Called frequently enough to catch overflows (<6.8s at 210 RPM max)
state pointer valid and writable

Postcondition

state->total_count updated with overflow-corrected accumulation
state->last_raw_count updated with current hardware counter value
state->revolutions calculated from total_count
state->position_deg calculated from total_count
Ready for next read (call again at next control loop iteration)

Invariant

total_count accuracy maintained across overflow boundaries
Counter wraps are detected and corrected
Bidirectional motion tracked correctly

Note

Thread Safety: Not thread-safe for same channel. Protect with mutex if needed.

Re-entrancy: Not reentrant for same channel. Safe for different channels.

Performance: ~2.5us execution, suitable for 250 Hz control loops

Memory: No dynamic allocation, stack-only

Warning

Must call frequently (>0.15 Hz) to prevent missed overflows

Do not skip reads for extended periods (>6 seconds at max speed)

Missed overflow causes +/-65536 count error (permanent until reset)

Attention

Call at consistent rate (e.g., 250 Hz) for accurate velocity measurement

Basic Usage Example:

```
#include "rx_mtu_encoder.h"

// Read encoder in 250 Hz control loop
void control_loop_250hz(void)
{
    rx_encoder_state_t state;
    rx_err_t err = rx_encoder_read_count(k_rx_mtu_channel_1, &state);

    if (err == k_rx_ok) {
        printf("Count: %ld, Position: %.2fdeg, Revolutions: %ld\n",
               state.total_count, state.position_deg, state.revolutions);
    } else {
        rx_log_error("ENCODER", "Read failed: %d", err);
    }
}
```

Velocity Calculation Example:

```
static int32_t last_count = 0;
static bool first_read = true;

void calculate_velocity_250hz(void)
{
    rx_encoder_state_t state;
    rx_encoder_read_count(k_rx_mtu_channel_1, &state);

    if (!first_read) {
        int32_t delta_counts = state.total_count - last_count;
        float dt_sec = 0.004f;  // 250 Hz = 4ms period
        float velocity_rps = (float)delta_counts / (1364.0f * dt_sec);
        float velocity_rpm = velocity_rps * 60.0f;

        printf("Velocity: %.2f RPM\n", velocity_rpm);
    }

    last_count = state.total_count;
    first_read = false;
}
```

Error Handling Example:

```
rx_encoder_state_t state;
rx_err_t err = rx_encoder_read_count(k_rx_mtu_channel_1, &state);

switch (err) {
    case k_rx_ok:
        // Success - use state data
        break;

    case k_rx_err_null_ptr:
        rx_log_error("ENCODER", "NULL state pointer");
        return;

    case k_rx_err_invalid_state:
        rx_log_error("ENCODER", "Not initialized");
        // Re-initialize encoder
        rx_encoder_init(&config);
        break;

    default:
        rx_log_error("ENCODER", "Unexpected error: %d", err);
        break;
}
```

Multiple Encoders Example:

```
// Read all 4 motor encoders
void read_all_encoders(void)
{
    const rx_mtu_channel_t channels[4] = {
        k_rx_mtu_channel_1, k_rx_mtu_channel_2,
        k_rx_mtu_channel_3, k_rx_mtu_channel_4
    };

    rx_encoder_state_t states[4];

    for (uint8_t i = 0; i < 4; i++) {
        rx_err_t err = rx_encoder_read_count(channels[i], &states[i]);
        if (err != k_rx_ok) {
            rx_log_error("ENCODER", "Motor %d read failed", i);
            states[i].total_count = 0;  // Use safe default
        }
    }

    // Now states[] contains all encoder positions
}
```

See also

rx_encoder_init() Must call before reading

rx_encoder_read_velocity() Alternative for velocity-only reads

rx_encoder_reset() Zero encoder without losing initialization

rx_encoder_state_t Structure updated by this function

Since

Version 1.0.0

Test test_rx_mtu_encoder.c::test_encoder_overflow() Tests overflow detection

NASA Power of 10 Compliance:

- Rule 5: 4 preconditions, 5 postconditions [OK]
- Rule 7: All return values checked by caller [OK]

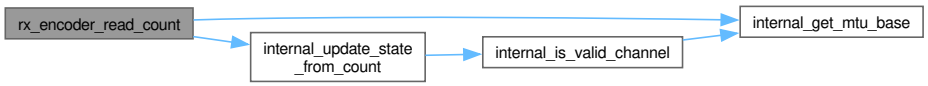
Read encoder count with automatic 16-bit overflow handling.

Definition at line 732 of file rx_mtu_encoder.c.

```
00733 {
00734   RX_VALIDATE_PTR(state, s_tag, "state pointer is nullptr");
00735
00736   volatile rx_mtu_channel_regs_t* const mtu = internal_get_mtu_base(channel);
00737   if (mtu == nullptr) {
00738     return k_rx_err_invalid_arg;
00739   }
00740
00741   RX_VALIDATE_INIT(s_encoder_initialized[channel], s_tag, "Encoder not initialized");
00742
00743   /* Read current counter value */
00744   const uint16_t current_count = mtu->tcnt;
00745
00746   return internal_update_state_from_count(state, channel, current_count);
00747 }
```

References internal_get_mtu_base(), internal_update_state_from_count(), s_encoder_initialized, and s_tag.

Here is the call graph for this function:



5.3.3.10 rx_encoder_read_raw()

```
rx_err_t rx_encoder_read_raw (
                const rx_mtu_channel_t channel,
                uint16_t * count)   [nodiscard]
```

Read raw encoder count.

Returns the current 16-bit counter value. Does not handle overflow - use rx_encoder_read_count() for overflow handling.

Parameters

in	channel	MTU channel
out	count	Pointer to store raw count (0-65535)

Returns

k_rx_ok on success

k_rx_err_null_ptr if count is nullptr

k_rx_err_invalid_arg if channel is invalid

k_rx_err_invalid_state if encoder not initialized

Read raw encoder count.

Reads the current value of the MTU TCNT (Timer Counter) register without any processing. This provides the raw 16-bit hardware count which wraps at 65536. For position tracking with wraparound handling, use rx_encoder_read_count() instead.

Use Cases:

- Debugging: Verify hardware is counting encoder edges
- Diagnostics: Check for stuck counter (hardware fault)
- Testing: Validate encoder wiring and signal quality
- Low-level access: When software wraparound tracking not needed

Hardware Behavior:

- Counter increments on forward encoder motion (Phase A leads Phase B)
- Counter decrements on reverse encoder motion (Phase B leads Phase A)
- Counter wraps: 65535 -> 0 (forward) or 0 -> 65535 (reverse)
- No software processing applied to value

Precondition

channel must be initialized via rx_encoder_init()

count must point to valid uint16_t variable

Postcondition

On success: *count contains current TCNT register value

Hardware counter unchanged (read-only operation)

Note

Thread-safe for read-only access (atomic 16-bit register read)

Very fast operation (~200 ns) - single register read

Does NOT handle wraparound - use rx_encoder_read_count() for that

Performance:

- Execution time: ~0.5 us @ 240 MHz
- Safe to call at very high frequency (> 10 kHz tested)

Example - Diagnostic Check:

```
uint16_t raw_count;
rx_err_t err = rx_encoder_read_raw(k_mtu_channel_1, &raw_count);
if (err == k_rx_ok) {
    rx_log_info("DIAG", "Raw counter: %u", raw_count);

    // Check if encoder is moving
    tx_thread_sleep(10);  // Wait 10 ms
    uint16_t new_count;
    rx_encoder_read_raw(k_mtu_channel_1, &new_count);
    if (new_count == raw_count) {
        rx_log_warn("DIAG", "Encoder not moving - check wiring");
    }
}
```

See also

rx_encoder_read_count() Read position with wraparound handling

rx_encoder_init() Must call before reading

Since

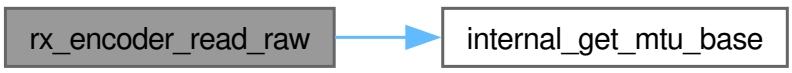
Version 1.0.0

Definition at line 714 of file rx_mtu_encoder.c.

```
00715 {
00716   RX_VALIDATE_PTR(count, s_tag, "count pointer is nullptr");
00717
00718   volatile rx_mtu_channel_regs_t* const mtu = internal_get_mtu_base(channel);
00719   if (mtu == nullptr) {
00720     return k_rx_err_invalid_arg;
00721   }
00722
00723   RX_VALIDATE_INIT(s_encoder_initialized[channel], s_tag, "Encoder not initialized");
00724
00725   *count = mtu->tcnt;
00726   return k_rx_ok;
00727 }
```

References internal_get_mtu_base(), s_encoder_initialized, and s_tag.

Here is the call graph for this function:



5.3.3.11 rx_encoder_read_velocity()

```
rx_err_t rx_encoder_read_velocity (
                float * velocity_rps,
                const float delta_time_s,
                const rx_mtu_channel_t channel)
```

Read encoder velocity.

Calculates velocity based on count change over time period. Assumes periodic calls at known rate (e.g., 250Hz control loop).

NOTE: Parameter order changed to prevent accidental swapping of channel/delta_time_s.